

Coordinated Malicious Behavior Detection in Blockchain Networks Using Temporal Graph Motifs

Bachelor Thesis

Sebastian Eduard Bruckner-Hrubesch
h12220272

Department of Information Systems and Operations Management
Institute for Complex Networks

Advisor: Mag. Ema Kahr, PhD

11th of October 2026

Table of Contents

List of Figures vii

List of Tables ix

List of Abbreviations ix

1 Introduction 2

2 Related Work 4

2.1 Graph-Based Detection of Malicious Behaviour . . . 4
2.2 Network Embeddings and Time-Aware Learning . . . 5
2.3 Datasets for Blockchain Transaction Analysis . . . 5
2.4 Temporal Motifs and Local Role Representations . . . 6
2.5 Position of This Thesis . . . 8

3 Theoretical Foundations 9

3.1 Blockchain and Ethereum fundamentals . . . 9
3.2 Graph-theory foundations . . . 9
3.3 Temporal networks . . . 10
3.4 Temporal motifs . . . 12
3.4.1 Counting algorithm . . . 12
3.4.1.1 Static motif counting . . . 13
3.4.1.2 Temporal motif counting . . . 14
3.4.2 Motif fingerprint . . . 14
3.5 Blockchain Relevance . . . 16
3.5.1 Types of Coordinated Malicious Behaviour on Ethereum . . . 17

4 Method 23

4.1 System overview . . . 23
4.2 Data acquisition . . . 24
4.2.1 Real-world datasets . . . 26
4.2.2 Validation templates and null models . . . 28
4.3 Temporal graph construction . . . 29
4.3.1 Graph model . . . 29
4.3.2 Building the graph from the data . . . 29
4.3.3 The non-motif baseline . . . 30
4.4 Temporal motif extraction . . . 31

4.4.1 Subgraph counting and retained local roles . . . 31
4.4.2 Ordering transfers with equal timestamps . . . 32
4.4.3 Fingerprint representations . . . 32
4.4.4 Temporal-window sweep . . . 33
4.5 Detection methods . . . 34
4.5.1 The three feature views . . . 34
4.5.2 Classifiers for RQ1 and RQ2 . . . 34
4.5.3 Classifier for RQ3 and RQ4 . . . 35
4.5.4 Handling class imbalance . . . 36
4.6 Evaluation . . . 36
4.6.1 Scope . . . 36
4.6.2 Cohorts and splits . . . 36
4.6.3 Metrics and window selection . . . 37
4.6.4 Statistical tests . . . 38
4.6.5 Robustness to event ordering . . . 39
4.7 Implementation details . . . 39
4.7.1 Language and libraries . . . 39
4.7.2 Project structure and entry points . . . 39
4.7.3 Reproducibility . . . 41
4.7.4 Result artifacts . . . 43

5 Results **45**
5.1 Evaluation Samples . . . 45
5.2 RQ1: Incremental Signal of Temporal Motif Fingerprints . . . 46
5.3 RQ2: Sensitivity to the Temporal Window . . . 48
5.4 RQ3: Distinct Motif Fingerprints per Category . . . 50
5.5 RQ4: Category-Specific Temporal Windows . . . 53
5.6 Additional Checks . . . 54
5.7 Answers to the Research Questions . . . 55

6 Discussion **56**
6.1 Interpretation of the Main Findings . . . 56
6.1.1 Detection value of temporal motifs . . . 56
6.1.2 Choice of the temporal window . . . 57
6.1.3 Differences between behaviour categories . . . 58
6.2 Theoretical Implications . . . 59
6.3 Practical Implications . . . 60
6.4 Limitations . . . 61

7 Conclusion & Outlook **63**
7.1 Conclusion . . . 63
7.2 Outlook . . . 64

Appendix **66**

A Engineering challenges 66
A.1 Locating the bottleneck 66
A.2 Alternatives considered 66
A.3 Long-run execution 67

References **67**

List of Figures

3.1 All 36 two- and three-node, three-edge δ-temporal motifs, indexed as $M_{i,j}$ by row i and column j. The first edge in each motif originates from the green node and edge labels denote temporal ordering. Figure from Paranjape et al. (2017). 13
3.2 Ponzi candidate pattern 18
3.3 Phishing transfer sequence 18
3.4 Pump-and-dump transfer sequence 19
3.5 Rug pull transfer sequence 19
3.6 Money laundering transfer sequence 20
3.7 Giveaway transfer sequence 20

4.1 The eight steps of the pipeline. The first two make up the preparation phase. The remaining six cover graph construction, fingerprint extraction, detection, analysis, and reporting in the execution phase. 25
4.2 The locked Results pipeline. The preparation phase fixes the cohorts and writes the locked artifacts, and the execution phase then computes the local role counts, trains the models, and runs the statistical tests. Italic labels name the source modules in `src/experiments`. 42

5.1 RQ1 feature comparison in the 40-role HuggingFace analysis using the default timestamp ordering. The controlled cohort is shown on the left and the prevalence cohort on the right. 46
5.2 RQ2 temporal-window sensitivity in the 40-role HuggingFace analysis using the default timestamp ordering. The controlled cohort is shown on the left and the prevalence cohort on the right. The top row reports held-out average precision with shaded 95 % bootstrap confidence intervals. The bottom row reports fingerprint coverage across all addresses in each cohort. 49
5.3 RQ3 category fingerprints using the globally selected two-week window and the default timestamp ordering. The two panels show the 40 local role components that vary most between categories in the development data, divided into two groups of 20 for readability. These components are selected from the five layers of 32 roles each, not from the 40-role HuggingFace catalogue. 51
5.4 RQ3 confusion matrix for the held-out Real-CATS addresses at the shared two-week window under the default timestamp ordering. Cells report numbers of addresses. 52

5.5 RQ4: mean development-CV average precision for each category across the nine temporal windows using the default timestamp ordering. 54

List of Tables

3.1 The terms used to describe temporal patterns and address features. The counted patterns are candidates, without a claim of statistical overrepresentation. . . . 15

3.2 The nine exact Real-CATS categories evaluated in RQ3 and RQ4 and their hypothesized temporal-network interpretation. . . . 22

4.1 The real-world datasets and their roles. The malicious share is computed over addresses. Synthetic validation templates are described separately in Section 4.2.2. . . . 28

4.2 The non-motif baseline features used for RQ1. . . . 30

4.3 Retained structural motif classes (after Paranjape et al. (2017)) and their behavioural interpretation in Ethereum transaction networks. Each class covers several δ-temporal roles that differ in the direction and order of the three transfers. . . . 32

4.4 The three models used in the detection experiments. . . . 35

4.5 The main libraries used by the pipeline. . . . 40

5.1 The three disjoint HuggingFace confirmation samples. The source contains 114,739 labelled targets. 39,332 pass the common activity and upper-tail rules before prior-run exclusions. . . . 45

5.2 RQ1: average precision of each feature view at the selected window for three disjoint confirmations (canonical ordering), the combined-minus-baseline difference with its 95 % confidence interval, the permutation p-value, and the decision. . . . 46

5.3 Prespecified secondary RQ1 representation comparisons at the development-selected window under canonical ordering. The first contrast tests fingerprint shape beyond total motif count. The second tests the combined model against a baseline augmented with motif composition and volume. P-values use Benjamini–Hochberg correction within the prespecified comparison family for each run. . . . 47

5.4 Final RQ1 triangle-role ablation at the selected 30-day window under canonical ordering. Corrected p-values use the prespecified representation-comparison family. . . . 48

5.5 Independent binary replication on the activity-matched Real-CATS sample under canonical ordering. . . . 48

5.6 RQ2: development-CV window selections for three disjoint confirmations. The final column states whether the selected maximum was uniquely superior after corrected comparisons under all seven orderings. 48
5.7 RQ3: overall separability of the criminal categories from their motif fingerprints (canonical ordering). 50
5.8 RQ3: one-vs-rest average precision for each behaviour category at the globally selected window (canonical ordering). The address count and class share are included because average precision depends on class prevalence. . 53
5.9 RQ4: held-out comparison of development-selected category-specific windows with one global window (canonical ordering). 53
5.10 RQ4: category-specific windows compared with the global window (canonical ordering). P-values use Benjamini–Hochberg correction. 55
5.11 Summary of the answers to the four research questions. 55

Abstract

Malicious activity on Ethereum can involve transaction sequences that are difficult to recognise from individual address summaries. This thesis examines whether temporal motif fingerprints improve malicious-address detection, how their usefulness depends on the temporal window, and whether they distinguish reported behaviour categories. The fingerprints count each address's local roles in ordered three-transfer candidate patterns. They are evaluated on two labelled Ethereum datasets using held-out addresses, nine windows from five minutes to 30 days, and seven orderings of transfers with equal timestamps.

Adding fingerprints to 17 conventional transaction and graph features increases average precision in all six cohort comparisons on the main dataset, but only three increases are statistically significant. The independent binary comparison also shows no significant improvement. Larger windows generally increase both average precision and fingerprint coverage, without establishing a unique optimum. Across nine malicious-behaviour categories, fingerprints contain statistically detectable category information, but classification remains limited, with macro average precision of 0.1953. Selecting separate windows for the categories gives no significant improvement over a shared window.

The results show that temporal motif fingerprints describe behavioural differences, but do not consistently add predictive value beyond conventional features. The evaluation concerns retrospective address classification, not direct detection of independently verified coordinated campaigns.

1. Introduction

Blockchain networks have become a critical part of the global financial infrastructure. As they have grown, they have also become an attractive target for criminal actors. According to the Chainalysis 2025 Crypto Crime Report, more than $40.9 billion flowed to illicit cryptocurrency addresses in 2024 alone (Chainalysis, 2025; Shi et al., 2025). This number is likely a lower bound. Chainalysis notes that annual illicit-volume estimates have grown by an average of 25% between reporting periods since 2020, so the 2024 total could exceed $51 billion (Chainalysis, 2025). The abuse ranges from ransomware and phishing scams to large-scale money laundering, and it affects both the Bitcoin and the Ethereum ecosystem (Sanjalawe and Al-Emari, 2023; Shi et al., 2025).

The harder problem is not the size of these losses, but the way they are produced. Sophisticated attackers do not rely on a single, continuously active address. Instead, they spread their activity over many short-lived wallets, where each wallet makes only a few transactions that look unremarkable on their own. In fact, 90.70% of criminal Bitcoin addresses carry a zero balance and 60% of them stop being active within 31 days (Shi et al., 2025). This behaviour deliberately imitates ordinary wallets, so a single transaction rarely looks suspicious. The signal of coordinated activity therefore does not appear in individual transactions, but in how those transactions are connected across several addresses and over time. A ransomware payment chain, a phishing fan-out or a layering operation only becomes visible once its transactions are considered together, in order, and across all the addresses involved.

Detecting these patterns requires a representation of how transactions connect addresses over time. Feature-engineering methods summarise transaction histories through statistics such as transaction count, address lifetime and balance variance (Al-Emari et al., 2021; Chen and Fu, 2022). These summaries do not explicitly count ordered interactions involving several addresses. Other approaches do incorporate time, through timestamp-aware network embeddings or models of changing transaction neighbourhoods (Ghosh et al., 2025; Wu et al., 2022b). Temporal motif features have also been applied to Ethereum phishing detection (Wang et al., 2023). The question is therefore not whether time or motifs have been used before, but whether a local motif-role fingerprint adds predictive information beyond conventional address features under the evaluation settings considered here.

Temporal motif fingerprints are a natural way to address this gap. They describe how an address participates in short, ordered transaction patterns within a bounded time window (Kovanen et al., 2011; Paranjape et al., 2017). Before comparison with a random-network reference, these patterns are candidate motifs. A motif in the statistical sense is a pattern that occurs significantly more often than expected under such a reference. This thesis counts candidate patterns and evaluates their usefulness for supervised detection. The term *motif fingerprint* names this representation and does not imply that its components have passed a test of overrepresentation. Earlier work on Bitcoin found cyclic triangle patterns

to be more frequent than in the communication networks examined (Paranjape et al., 2017). Whether per-address fingerprints improve the supervised detection of malicious behaviour remains the question addressed here.

This thesis investigates the following research questions (RQs):

RQ1. Do temporal motif fingerprints improve the detection of malicious Ethereum addresses beyond transaction-based aggregates and static graph features?

RQ2. How does the temporal window size (δ) affect the usefulness of temporal motif fingerprints for detecting coordinated malicious behaviour?

RQ3. Do different categories of coordinated malicious behaviour exhibit distinct temporal motif fingerprints?

RQ4. Does selecting a separate temporal window for each behaviour category improve category classification compared with using one shared window?

The remainder of this thesis is organised as follows. Chapter 2 reviews graph-based detection methods, the available transaction datasets, and previous work on temporal motifs. Chapter 3 introduces the blockchain and graph-theoretic concepts needed to define temporal patterns, local roles, and address fingerprints. Chapter 4 describes the data preparation, graph construction, feature extraction, and evaluation protocol. Chapter 5 reports the findings for the four research questions. Chapter 6 interprets these findings, relates them to earlier work, and discusses the limitations of the study. Chapter 7 summarises the conclusions and outlines directions for further research. Appendix A records the engineering challenges and implementation decisions behind the pipeline.

2. Related Work

Research on malicious blockchain activity differs in what is represented and what is predicted. Some methods classify addresses from transaction summaries, while others learn from their neighbourhoods or from ordered sequences of transfers. The datasets also differ in whether labels describe transactions, addresses, trust relations, or particular types of abuse. This chapter reviews these approaches and explains how the present evaluation relates to them.

2.1. Graph-Based Detection of Malicious Behaviour

A transaction graph provides more context than a list of independent payments. However, the choice of graph determines which interactions are visible. Lee et al. (2020) study several Ethereum interaction networks, covering ordinary transactions, contract interactions, and token transfers. Their comparison shows why the graph representation must be specified before its properties can be interpreted. This is relevant here because an ordinary transaction graph and a graph that also includes token transfers do not describe the same set of activities.

Early supervised detection methods use address-level summaries of such networks. Ostapowicz and Żbikowski (2019) compare random forests, support vector machines (SVMs), and Extreme Gradient Boosting (XGBoost) on more than 300,000 Ethereum accounts. They also examine how individual features affect the classification. Their work establishes a practical starting point for the present thesis: conventional address features already contain useful detection information, so any additional representation needs to be tested against them.

Chen and Fu (2022) construct 149 features from an address's direct transactions, drawing on patterns associated with fraudulent bank accounts. Their method filters the features and uses a random forest for phishing detection, reporting 98.89% precision on their dataset. The features have a clear connection to transaction behaviour, but they summarise the target's activity rather than count ordered interactions among several addresses.

Sanjalawe and Al-Emari (2023) also derive features from a weighted directed multigraph. Their approach combines feature selection with a semi-supervised generative adversarial network and reports 95.06% detection accuracy. The selected features include structural measures and summaries of transaction timing. This is different from representing the exact order of transfers in a small connected subgraph. The reported scores from these studies cannot be compared directly with each other or with the present results, because the datasets, class proportions, and evaluation procedures differ.

Together, these studies motivate the conventional feature baseline in this thesis. Transaction counts, values, activity durations, and static graph properties can identify differences between malicious and benign addresses. The purpose of adding local motif-role counts is to test whether short transaction sequences contribute information beyond these summaries.

2.2. Network Embeddings and Time-Aware Learning

Network embeddings replace a manually designed feature vector with a learned representation of an address and its neighbourhood. Wu et al. (2022b) introduce trans2vec for Ethereum phishing detection. The method incorporates transaction amounts and timestamps into biased random walks and then applies a one-class support vector machine. It therefore already uses temporal information. Its representation is learned from walks, however, rather than expressed as separate counts of predefined local roles.

Time-aware graph modelling is also studied outside fraud classification. Lin et al. (2020) represent Ethereum records as a temporal weighted multidigraph and compare walk strategies that account for transaction times and amounts. Their evaluation concerns temporal link prediction. This supports the importance of retaining repeated and timestamped interactions, but link prediction is a different task from deciding whether an address is malicious.

Graph neural networks provide another way to combine an object's features with information from its neighbours. In the Bitcoin setting, Weber et al. (2019) compare graph convolutional networks with conventional classifiers on the Elliptic dataset. Random forests perform strongly in their experiments. The result is a useful reminder that a graph-based model does not automatically improve on a feature-based classifier, even when the underlying data is relational.

More recent Ethereum work models changing neighbourhoods explicitly. Ghosh et al. (2025) combine local and global temporal dependencies for phishing detection. Their model processes sequences of transaction neighbourhoods and learns how behaviour changes over time. This differs from a fixed catalogue of short event patterns, but it rules out a general claim that earlier blockchain detection methods ignore time.

The distinction relevant to this thesis is therefore between representations, not simply between methods that use time and methods that do not. Temporal motif fingerprints provide explicit counts for particular ordered patterns. Embeddings and neural models learn their representations from the data. The present experiments compare fingerprints with conventional features. They do not benchmark all of these learned alternatives or isolate the causal contribution of temporal ordering.

2.3. Datasets for Blockchain Transaction Analysis

A useful detection dataset needs both behavioural observations and labels for the objects being classified. It also needs sufficient transaction detail for the proposed representation. An address-level table of summary features cannot by itself reconstruct the order of interactions between several addresses.

Al-Emari et al. (2021) introduce the Benchmark Labelled Transactions of Ethereum (BLTE) dataset as a public resource for evaluating detection methods. The later study by Sanjalawe and Al-Emari (2023) describes its source as Ethereum Classic and uses abnormal-address information from EtherScamDB. BLTE is relevant as an established transaction-based

benchmark. Its source coverage and binary labels must nevertheless be distinguished from those of a mainnet Ethereum address-classification task.

The Elliptic dataset released by Weber et al. (2019) contains more than 200,000 Bitcoin transactions connected by directed payment flows. Its nodes represent transactions, not persistent Ethereum addresses. It supports the study of illicit transaction detection over time, but its graph construction and prediction unit differ from those used here.

The Real-world dataset of Cryptocurrency Addresses with Transaction Profiles (Real-CATS), introduced by Shi et al. (2025), combines 103,203 criminal addresses and 106,196 benign addresses across Bitcoin and Ethereum. Criminal labels are drawn from abuse reports, while benign addresses are collected from exchange customers. Alongside address profiles, the release contains transaction histories that support alternative feature extraction. Its Ethereum categories make it useful for the category comparisons in this thesis. These labels describe reported abuse, however, and do not establish that every address in a category belongs to one coordinated campaign. A supplementary dataset of cryptocurrency addresses and transaction profiles, named Sup-CATS, provides a separate deployment-oriented evaluation sample.

The Ethereum Fraud Dataset by Activity provides another source of transaction histories and address labels (fesevu, 2025). The release includes raw transfers with sender, receiver, timestamp, and transaction metadata, together with labels assembled from Etherscan and curated sources. This makes it possible to compute conventional features and temporal fingerprints from the same underlying observations. Its role in the present study is as a data resource, not as evidence for the performance of a particular detection method.

These resources are complementary rather than interchangeable. Their labels refer to different objects, and their transaction histories cover different parts of the underlying networks. Chapter 4 therefore separates the role of each source from the selection and graph-construction rules used in the experiments. Comparisons between feature sets use the same addresses and data splits, so that differences in their scores are not caused by changing the evaluation sample.

2.4. Temporal Motifs and Local Role Representations

Motif analysis begins with the idea that small patterns can reveal structure that is not captured by single-node statistics. Milo et al. (2002) identify network motifs through overrepresentation relative to randomised networks. Counting a candidate pattern and establishing that it is statistically overrepresented are therefore separate steps. McDonnell et al. (2014) make a further distinction between a global subgraph count and a node's participation in its local roles. Their motif-role fingerprints describe the positions occupied by individual nodes in directed patterns. This provides the basis for an address-level representation rather than a single summary of the entire transaction graph.

Kovanen et al. (2011) extend pattern analysis to event sequences whose similarity depends on both topology and temporal order. Paranjape et al. (2017) develop an efficient counting

framework for δ-temporal motifs and show that pattern counts and relevant time scales differ across networks. In this framework, an ordered sequence must fit within a bounded time span. The present thesis uses this counting definition while referring to untested patterns as candidate motifs. Their predictive value is evaluated without claiming network overrepresentation.

Several studies connect these ideas to financial behaviour. Lal et al. (2021) examine temporal cycles in Ethereum money trails and find different cyclic patterns associated with suspicious activities. Their heuristic analysis concerns paths through which funds move. It supports examining connected transfer sequences, but it is not the same evaluation as testing a fixed local-role fingerprint against a conventional feature baseline.

Wu et al. (2022a) combine temporal and attributed heterogeneous patterns with other transaction features to detect Bitcoin mixing services. They use positive and unlabelled learning because reliable negative labels are limited. This is direct evidence that motif-based features have already been used in supervised blockchain analysis. The target is membership in a mixing service, which is different from the binary and category labels considered here.

There is also a direct Ethereum precedent. Wang et al. (2023) study phishing detection using temporal motif features extracted from transaction subgraphs. The combination of blockchain data, labelled phishing addresses, and temporal motif features is therefore not new. The present work builds on this line of research through its evaluation of local-role fingerprints, temporal-window sensitivity, and differences between recorded behaviour categories.

Arnold et al. (2024) analyse three-transfer patterns on two Bitcoin datasets and one non-fungible token (NFT) transaction dataset. Their results show that global counts can be dominated by a small number of participants or short periods of activity. Local signatures and pattern completion times reveal information that these global totals hide. They also examine randomised timestamp references and the ambiguity of ordering transactions within a block. These observations motivate both the address-level fingerprints and the equal-timestamp sensitivity checks in this thesis. Their analysis is primarily descriptive, rather than a held-out comparison of malicious-address classifiers.

Outside blockchain networks, Liu et al. (2023) evaluate temporal motif features for fraud detection on Mercari transactions and a synthetic J.P. Morgan Chase network. They report improvements over simple graph features and the embedding methods Large-scale Information Network Embedding (LINE) and node2vec. Their Venmo analysis addresses different prediction and description tasks. These findings support the usefulness of temporal patterns in financial networks, but do not establish that the same improvement will hold for Ethereum addresses.

A different use of motifs appears in Li et al. (2023). The authors combine motif matrices and temporal graph convolutional learning on the Bitcoin-Alpha and Bitcoin over-the-counter (Bitcoin-OTC) signed trust networks. Their revised model predicts the signs of trust links. The motifs describe structure within snapshots, while temporal information

is combined across snapshots. These are neither Ethereum payment histories nor direct counts of ordered three-transfer occurrences, so the task and representation need to be kept separate from the present approach.

2.5. Position of This Thesis

The reviewed literature already contains time-aware embeddings, temporal graph models, and motif-based blockchain detection. The contribution of this thesis is an evaluation of a particular local motif-role representation under a common protocol. It compares conventional features, fingerprints alone, and their combination on the same held-out addresses. It also examines how the permitted time span affects the representation, whether recorded abuse categories are separable through fingerprints, and whether category-specific windows improve on a shared window.

These comparisons address the four research questions introduced in Chapter 1. The evaluation also records how the results change when transfers with equal timestamps are ordered differently. Its scope is the predictive usefulness and stability of the fingerprints under the available labels and graph coverage. It does not establish that temporal order alone causes an improvement or that the counted candidates are statistically overrepresented network motifs.

3. Theoretical Foundations

This chapter will cover the theoretical foundations that this thesis builds upon.

3.1. Blockchain and Ethereum fundamentals

First, we will cover what a Blockchain is and how Ethereum works and handles transactions. A blockchain is a distributed ledger in which transactions are grouped into blocks and linked chronologically so that later blocks reference earlier ones through cryptographic hashes (Buterin, 2014; Nakamoto, 2008). This makes the ledger append-only, because changing a past record would require rewriting the block and all of the blocks that follow under the network's consensus rules (Nakamoto, 2008). For this thesis, the important part is not the consensus mechanism itself, but that blockchain data is ordered, time-stamped, and persistent.

Ethereum extends this basic blockchain idea with account-based state and smart-contract execution (Buterin, 2014). The relevant transaction record can be represented as a tuple (s, r, v, t, g), where s is the sender, r the receiver, v the transferred value, t the timestamp, and g the gas usage (transaction fee) (Al-Emari et al., 2021; Sanjalawe and Al-Emari, 2023). Ethereum distinguishes between externally owned accounts (EOAs), which are controlled by private keys and initiate transactions directly, and contract accounts, which execute code when invoked (Al-Emari et al., 2021; Buterin, 2014).

Because multiple transactions can occur between the same pair of addresses at different times, with different values and gas costs, Ethereum is naturally modeled as a weighted temporal directed multigraph (Kovanen et al., 2011; Paranjape et al., 2017). This is the abstraction used in the remainder of the chapter. In contrast, Bitcoin follows an unspent transaction output (UTXO) model, where transactions consume and create unspent outputs rather than directly transferring value between persistent accounts (Nakamoto, 2008).

With this data model established, the next section introduces the formal graph representation.

3.2. Graph-theory foundations

A graph is a mathematical structure consisting of a set of nodes V and a set of edges E, where nodes represent entities and edges represent pairwise relations between them (Newman, 2010). In the context of this thesis, nodes correspond to Ethereum addresses and edges correspond to transactions between them.

Because transactions have a clearly defined sender and receiver, the appropriate structure is a *directed graph*, where an edge $(u, v) \in E$ represents a directed relation from node u to node v (Newman, 2010). Since the same pair of addresses may transact multiple times, each with a distinct value and timestamp, the data is more precisely represented as a

directed multigraph, in which multiple edges between the same ordered pair of nodes are permitted (Paranjape et al., 2017; Sanjalawe and Al-Emari, 2023).

For a node v, the *in-degree* $d^-(v)$ counts the number of incoming edges and the *out-degree* $d^+(v)$ counts the number of outgoing edges (Newman, 2010). These degree measures serve as basic node-level features and are used as baselines in the classification experiments in Chapter 4.

The directed multigraph provides the structural foundation for the graph model used throughout this thesis. The following section extends this static representation by incorporating the timestamps of transactions, which is necessary to capture the temporal ordering of interactions.

3.3. Temporal networks

The graph model introduced in Section 3.2 treats the Ethereum transaction graph as a static object once timestamps are discarded. This section explains why that is insufficient for detecting coordinated malicious behavior.

From static to temporal.

Static networks aggregate all interactions into a single, time-independent structure. While such aggregation is computationally efficient, it completely ignores the ordering of events, which is exactly the information that distinguishes coordinated sequential behavior from coincidental events. Temporal networks take this information into account by retaining the full sequence of timestamped edges (Kovanen et al., 2011; Paranjape et al., 2017).

Time-respecting paths and edge ordering.

A directed time-respecting path follows edges in both their transfer direction and their temporal order (Kovanen et al., 2011). With distinct timestamps, a path from v_0 to v_k is a sequence of the form

$$(v_0, v_1, t_1), \ldots, (v_{k-1}, v_k, t_k), \qquad t_1 < \cdots < t_k.$$

For example, a transfer $A \to B$ at 10:00 followed by $B \to C$ at 10:05 forms a time-respecting path from A to C. Reversing the times leaves the same static edges, but removes this path. The first ordering permits a relay interpretation, although it does not prove that the same funds were forwarded.

Transfers sharing a block timestamp need an additional ordering rule, since the timestamp alone does not determine which came first. The treatment of these ties is described in Section 4.4. Candidate temporal patterns are not restricted to paths. Their edges must form a connected graph when edge directions are ignored, but a star with only incoming transfers is also a valid candidate structure (Paranjape et al., 2017).

The time window δ.

Temporal order alone does not limit how far apart the transfers may occur. This thesis follows the total-span condition of Paranjape et al. (2017). For a candidate sequence of l edges, the first and last timestamps must satisfy

$$t_l - t_1 \leq \delta.$$

The bound applies to the whole sequence, not separately to each gap between successive edges.

A sliding window enforces this bound as the ordered transfers are processed. For a new transfer at time t, earlier transfers before $t - \delta$ can no longer form a valid candidate ending at that transfer. The interval $[t - \delta, t]$ therefore moves with the event sequence. These windows overlap rather than dividing the history into separate fixed bins. A candidate may cross an hour or day boundary as long as its total span remains within δ (Paranjape et al., 2017).

Small windows restrict candidates to short bursts, while larger windows admit interactions spread across days or weeks. No single interval is assumed to suit every behaviour. This thesis compares nine windows from five minutes to 30 days, with the grid given in Section 4.4.4. The window is selected on development data without using the held-out test results, as described in Section 4.6. Its value limits the span of a three-transfer candidate, not the duration of a malicious campaign.

What static projection loses.

The aggregated static graph loses two things: the *ordering* of edges and the *temporal density* of interactions. This means that two structurally identical subgraphs in the static graph could correspond to entirely different behaviours depending on whether the underlying transactions occurred rapidly or were spread across a bigger time span. Kovanen et al. (2011) observe that static networks are often time-aggregates of systems where connections are not continuously active, and that temporal inhomogeneities, which static representations do not capture, have an important effect on dynamic processes. In the Ethereum context, a fan-out pattern in the static graph (one address sending to many others) looks the same regardless of whether all transfers happened within ten seconds or over ten months. Only the temporal graph reveals whether the fan-out was a coordinated event.

Motivation for temporal motifs.

The observation that edge ordering matters and that meaningful patterns must occur within a bounded time window motivates the concept of temporal motifs introduced in Section 3.4. Rather than asking which subgraph topologies are over-represented in the static graph, temporal motifs ask which *ordered sequences* of interactions recur within a window δ. This is the appropriate question for detecting coordinated behavior in blockchain networks, where the timing of transactions is as informative as their structure.

3.4. Temporal motifs

Static network motifs.

The concept of network motifs was introduced by Milo et al. (2002) to describe small, recurring subgraph patterns that appear significantly more often in a real network than in a randomised reference. These patterns are considered the building blocks of network structure, so that different classes of networks (biological, social, technological) tend to be characterised by different sets of over-represented motifs (Milo et al., 2002). In static graphs, a motif is formally defined as a connected induced subgraph on k nodes whose frequency in the observed network exceeds what would be expected by chance under a suitable null model (Milo et al., 2002; Newman, 2010).

δ-temporal motifs.

Paranjape et al. (2017) extend the static motif concept to temporal networks by introducing δ-temporal motifs. A k-node, l-edge δ-temporal motif is an ordered sequence of l timestamped edges involving k nodes. The graph formed by these edges must be connected, and the time between the first and last edge must be at most δ. Two occurrences represent the same motif when their nodes can be relabelled without changing the direction or temporal order of the edges (Paranjape et al., 2017).

Candidate patterns and statistical motifs.

The temporal counting framework defines patterns by their structure, edge order, and time span. Counting an occurrence does not itself establish that the pattern is more common than expected by chance. In this thesis, an untested pattern is therefore a *candidate motif*, or a temporal subgraph. The terms *motif role* and *motif fingerprint* refer to the features derived from these candidates. Statistical overrepresentation would require a separate comparison with randomised networks under a specified null model (Milo et al., 2002). The supervised tests used here assess prediction and category separability, not network overrepresentation.

The catalogue of δ-temporal motifs.

For motifs with three edges on two or three nodes, Paranjape et al. (2017) enumerate a complete catalogue of 36 patterns. It contains four *2-node motifs*, in which all three edges occur between the same pair of nodes, 24 *star motifs*, in which one centre node interacts with two distinct neighbours, and eight *triangle motifs*, in which every pair of the three nodes is connected. Only two of the eight triangle motifs form a directed cycle. The individual patterns also differ in the direction and temporal ordering of their edges. The full catalogue is illustrated in Figure 3.1.

3.4.1. Counting algorithm

Motif counting algorithms form the computational foundation of any motif-based analysis. This section first outlines the established methods for static graphs and then explains why a fundamentally different approach is required for temporal data.

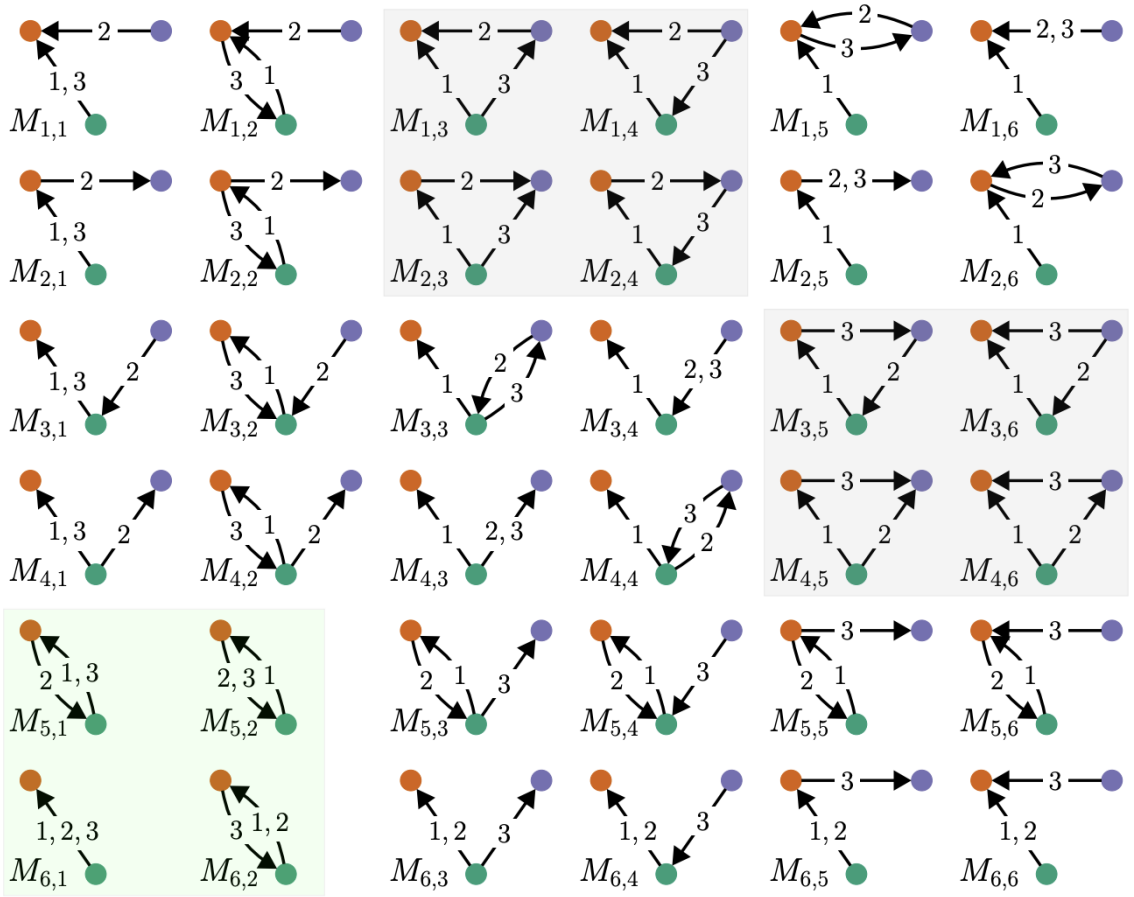


Figure 3.1.: All 36 two- and three-node, three-edge δ-temporal motifs, indexed as $M_{i,j}$ by row i and column j. The first edge in each motif originates from the green node and edge labels denote temporal ordering. Figure from Paranjape et al. (2017).

3.4.1.1. Static motif counting

Static graph motif counting is a well-studied problem. Given a graph G, the goal is to count how many times a given subgraph pattern, such as a triangle or a directed cycle, appears in G. Algorithms for this task range from exact neighborhood-intersection methods to matrix-based approaches and sampling-based approximations, covering a broad spectrum of accuracy and efficiency trade-offs (Yin et al., 2025). These methods are computationally mature and underpin many downstream tasks such as clustering coefficient computation and community detection.

However, static methods operate on a time-aggregate representation: all edges that ever existed between two nodes collapse into a single connection, regardless of when they occurred (Kovanen et al., 2011). This means that the temporal ordering is lost entirely. Paranjape et al. (2017) show that even when a triangle is topologically present in a network, only a subset of its instances are temporally valid. The others either occur in a different order or fall outside a given time window δ. A static counter treats all of them as identical. For blockchain transaction networks, this distinction is critical because coordinated malicious behaviour happens through transaction sequences that have a specific temporal order, something that static aggregation cannot recover.

Algorithm 1 δ-Temporal Motif Counting adapted from Paranjape et al. (2017)

Require: Time-sorted sequence $S = (e_1, \ldots, e_L)$, window δ, motif length l
Ensure: Count of each l-edge δ-temporal motif in S

```
left ← 1
patternCounts ← ∅
for right = 1, ..., L do
    while t_right − t_left > δ do
        RemoveFromWindow(e_left)
        left ← left + 1
    end while
    AddToWindow(e_right)
end for
return patternCounts
```

Procedure RemoveFromWindow(e) *// expire e and withdraw prefix contributions*

```
patternCounts[e] ← patternCounts[e] − 1
for all suffix σ of length < l − 1 in patternCounts do
    patternCounts[(e, σ)] −= patternCounts[σ]
end for
```

Procedure AddToWindow(e) *// admit e and extend all existing prefixes*

```
for all prefix π of length < l in patternCounts, longest first do
    patternCounts[(π, e)] += patternCounts[π]
end for
patternCounts[e] ← patternCounts[e] + 1
```

3.4.1.2. Temporal motif counting

Temporal motif counting extends the static counting problem by requiring that all edges of a motif occur in a specific order within a bounded time window δ.

The general counting algorithm proceeds in three steps: (1) identify all static instances of the subgraph H in the static graph G, (2) for each instance, gather all corresponding temporal edges into a time-sorted sequence S, and (3) process S within a sliding window of size δ, tracking how often each ordered edge pattern appears within that window (Paranjape et al., 2017). Algorithm 1 formalizes this procedure.

For 2-node motifs this runs in $O(m)$ time, which is optimal. For 3-node, 3-edge star and triangle motifs, specialized algorithms reduce the worst-case complexity significantly, with the fast triangle algorithm achieving up to a 56.5× speedup over the general method on the Bitcoin transaction network (Paranjape et al., 2017). This scalability is relevant to the present work, as blockchain transaction graphs are large and the motifs we are interested in correspond directly to 3-node star and triangle patterns within a short time window.

3.4.2. Motif fingerprint

The identification of recurring subgraph patterns in a network provides insight into the local structural properties of individual nodes. A global count records how often a pattern

occurs anywhere in the graph. It does not say which position an individual node occupies. The *motif-role fingerprint* of McDonnell et al. (2014) retains this distinction by collecting counts for the local roles of a focal node, such as an initiator, intermediary, or receiver.

McDonnell et al. (2014) further show that functional motif-role fingerprints, obtained from counts of partial subgraphs, can be transformed into structural fingerprints by multiplying them with an invertible transformation matrix, establishing a mathematically well-defined correspondence between the two representations. The authors identify anomaly detection and the identification of structurally similar nodes as primary use cases, which makes it well applicable for this thesis.

The concept is extended here to temporal graphs following the δ-temporal counting framework. A global candidate pattern first defines the ordered transfers. Choosing a focal address then determines its local role in that occurrence. Counts of these roles form the address fingerprint. Table 3.1 distinguishes these levels.

Table 3.1.: The terms used to describe temporal patterns and address features. The counted patterns are candidates, without a claim of statistical overrepresentation.

Term	**Example**	**Meaning**
Global candidate motif	$A \to B$, then $A \to C$, then $A \to B$	The complete ordered pattern, considered without selecting a focal address. A statistical motif additionally requires evidence of overrepresentation against a network null model.
Focal node	Address A	The address whose participation is being described.
Local motif role	A as the outgoing star centre	The position of the focal address, including the directions and ordering of the transfers relative to it.
Motif class	Star, all outgoing	A structural grouping of related patterns or roles. A class is not one feature column.
Address fingerprint	$\mathbf{f}_v^\delta$	The complete vector of local role counts for address v, with one component for each retained role.

For the full local catalogue used in this thesis, the raw fingerprint is defined as

$$\mathbf{f}_v^\delta = \left(c_{v,1}^\delta,\, c_{v,2}^\delta,\, \ldots,\, c_{v,40}^\delta\right) \tag{3.1}$$

where $c_{v,j}^\delta$ counts the number of candidate occurrences in which address v participates in local role j within a span of at most δ. The 40 components are local motif-role counts, not 40 global patterns or 40 motif classes. Raphtory reports 24 star-centre roles, eight two-node roles, and eight triangle roles. Section 4.4.1 explains their relationship to the 36 global patterns and the 32-role subsets used when triangle coverage is incomplete. A feature matrix has one row per address and one column per retained local role.

This serves two purposes. First, it reduces the high-dimensional transaction graph to a compact, interpretable, and fixed-size representation per address. Second, it encodes behavioral structure rather than raw transaction statistics because an address involved in coordinated activity is expected to exhibit a distinct fingerprint compared to ordinary users, as coordinated behavior by definition produces recurring, time-constrained interaction patterns among a group of addresses. The fingerprint therefore creates a behavioral signature of each address showing small recurring temporal subgraph patterns.

3.5. Blockchain Relevance

This section connects the foundations from the previous sections to the goal of this thesis, which is detecting coordinated malicious behaviour on Ethereum.

The abstractions introduced above are not only mathematically convenient, but they also match the structure of Ethereum transaction data. As shown in the section on Ethereum fundamentals and in Section 3.2, Ethereum transactions are directed interactions between addresses, and every interaction is recorded together with persistent metadata such as a timestamp, a value, and the gas usage. The consequence for this thesis is that Ethereum is not just a static graph, but a transaction graph that keeps changing over time.

This temporal aspect is not a minor detail. Prior work on Ethereum anomaly detection argues that the time-related nature of transactions makes transaction graphs well suited for temporal network analysis (Wittek et al., 2019). Studies of money trails report that activities such as phishing, gambling and money laundering show distinct cyclic transfer patterns that can only be identified when the order of transactions is preserved (Lal et al., 2021). Other work shows that suspicious behaviour often appears as rapid transaction bursts, sudden volume changes and short-window interaction patterns that a static graph cannot fully recover (Alomari and Cheng, 2023; Ghosh et al., 2025). Once the graph from Section 3.2 is extended with timestamps, it becomes possible to analyse not only *who* interacts with whom, but also *when*, *how often* and *in which order*.

As discussed in Section 3.3, a static aggregation can show that a group of addresses is connected, but it cannot tell a slow, unrelated sequence of transfers apart from a tightly coordinated one that runs within minutes. This matters because coordinated malicious activity is rarely a single transaction. It usually consists of several transfers spread across multiple addresses, for example to hide the origin of funds, redistribute them or imitate legitimate activity.

For this thesis, this has a direct methodological consequence. The graph-theoretic and temporal foundations from the previous sections turn raw transaction histories into higher-level behavioural patterns. Candidate temporal motifs describe recurring, time-local interaction structures, and local-role fingerprints make these structures measurable for each address. Earlier work already retains temporal information in Ethereum transaction networks (Lin et al., 2020). The step taken here is to evaluate an explicit local-role representation of that information, as set out in Section 2.5.

This leads to the next step. If Ethereum is treated as a temporal transaction graph, the question is no longer only how addresses are connected, but which forms of coordinated malicious behaviour produce characteristic temporal interaction patterns. The following section therefore looks at coordinated malicious behaviour on Ethereum and describes the main categories considered in this thesis.

3.5.1. Types of Coordinated Malicious Behaviour on Ethereum

The Ethereum blockchain hosts a broad range of fraudulent and malicious activities, made possible by the combination of pseudo-anonymity, programmable smart contracts, and the irreversibility of on-chain transactions. A taxonomy of cybercrime on the Ethereum blockchain derived from 5,644 verified malicious addresses identifies 19 distinct categories of fraud, with the total stolen funds across the observation period reaching $1.65 billion on-chain alone, a figure that exceeds self-reported losses to the United States Federal Trade Commission by a factor of almost 16, underscoring the scale of underreporting (Hornuf et al., 2025). Two sets of categories are distinguished below. The empirical comparison in RQ3 and RQ4 uses the nine exact Real-CATS labels that pass the minimum-size rule: Blackmail Scam, Contract Exploit Scam, Fake Returns Scam, Hack Scam, Impersonation Scam, NFT Airdrop Scam, Phishing Scam, Pigbutchering Scam, and Rug Pull Scam (Shi et al., 2025). Ponzi schemes, pump-and-dump schemes, money laundering, and giveaway scams remain useful examples of temporal mechanisms, but they are not treated as inferential Real-CATS classes in this thesis. Real-CATS labels reported behaviour types rather than independently verified coordination events, so coordination is a proposed network interpretation, not an additional ground-truth label.

Ponzi Schemes.

Ponzi schemes are the largest category of on-chain cybercrime by aggregate stolen value, accounting for approximately 60% of all funds transferred to verified malicious addresses on Ethereum (Hornuf et al., 2025). Operators promise high returns to investors and use incoming deposits from new participants to pay earlier ones, creating the appearance of a functioning investment operation (Bartoletti et al., 2020). On Ethereum, Ponzi logic is often encoded directly in smart contracts, which execute the redistribution of funds automatically and transparently (Chen et al., 2020a). Funds may therefore flow from many participant addresses into a central contract and later be redistributed outward, producing plausible fan-in and fan-out candidate patterns. Real-CATS has no exact Ponzi label, however, and the thesis does not infer Ponzi membership from another category such as Investment Scam or Fake Returns Scam.

Phishing.

Phishing attacks involve deceiving users into approving malicious smart contract interactions or transferring funds to attacker-controlled addresses (Chen et al., 2020b). In the Ethereum context, phishing has evolved beyond simple address impersonation to include signature-based token approval theft, where a victim signs a transaction that grants the

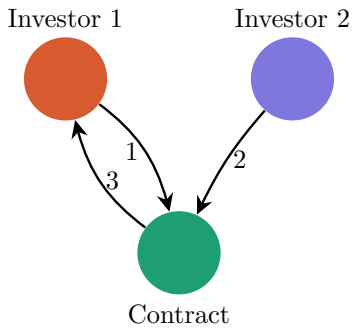


Figure 3.2.: Illustrative Ponzi sequence. Two deposits arrive (edges 1, 2). The payout to the earlier investor (edge 3) follows a fresh deposit, so $t_1 < t_2 < t_3$.

attacker unlimited transfer rights over their tokens (Hornuf et al., 2025). The coordination aspect manifests in the use of drainer contracts: a central attacker-controlled contract collects funds from many victims in rapid succession, often within short time windows, producing a repeated fan-in temporal pattern anchored at the malicious contract address.

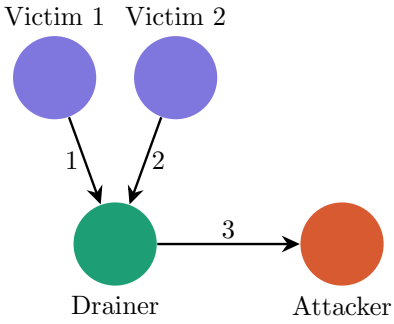


Figure 3.3.: Phishing drainer. Victims are drained into a central contract within a tight window (edges 1, 2), which then sweeps the pooled funds to the attacker (edge 3).

Pump-and-Dump Schemes.

Pump-and-dump schemes involve a coordinated group of addresses accumulating a low-liquidity token, artificially inflating its price through simultaneous buy activity, and then selling their holdings once uninformed traders enter the market (Hornuf et al., 2025; Li et al., 2021). The coordination is explicit and time-critical: the buy phase and the sell phase each involve multiple addresses acting within bounded periods in order to maximize price impact and exit before the market corrects. Such activity could produce dense, time-local transaction patterns, but Real-CATS has no pump-and-dump label. Its temporal window is therefore not estimated in RQ4.

Rug Pulls and Exit Scams.

A rug pull occurs when the developers of a token project attract liquidity, typically by depositing token reserves into a decentralized exchange, and then abruptly withdraw all deposited funds, rendering the token worthless (Hornuf et al., 2025; Xia et al., 2021). Exit scams follow a similar pattern at the level of initial coin offerings: funds raised

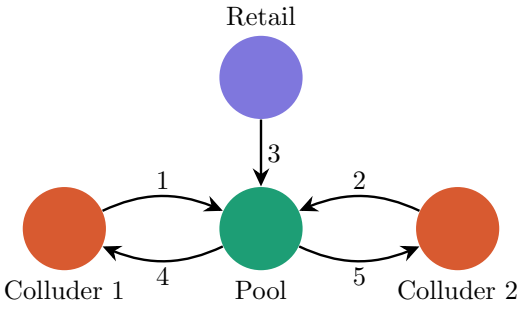


Figure 3.4.: Pump-and-dump. Each colluder buys (edges 1, 2) and later sells (edges 4, 5) against the same pool, with the retail victim's buy (edge 3) in between: $\{1, 2\} \prec 3 \prec \{4, 5\}$.

from investors are transferred out of the project contract to attacker-controlled addresses immediately or within a short period after fundraising closes. Both patterns produce a concentrated outflow from a contract address to a small set of beneficiary addresses within a narrow time window, a temporal interaction structure that differs structurally from ordinary liquidity provision.

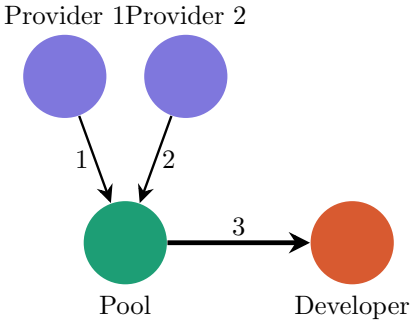


Figure 3.5.: Illustrative rug-pull sequence. Liquidity is added (edges 1, 2), then the pool is drained in a concentrated transfer (edge 3). The diagram does not prescribe which empirical window is optimal.

Money Laundering.

Money laundering on Ethereum typically involves moving illicitly obtained funds through a series of intermediate addresses to obscure their origin before depositing them at an exchange or converting them to a less traceable asset (Lal et al., 2021). This process, known as *layering*, can produce multi-hop transfer chains whose individual hops occur quickly even when the wider laundering operation lasts much longer. Temporal cycle analysis of Ethereum transactions has found cyclic transfer patterns associated with laundering activity (Lal et al., 2021), making it a candidate for temporal-motif analysis. Real-CATS has no money-laundering label, so this thesis does not estimate a laundering-specific window or compare it empirically with pump-and-dump activity.

Giveaway Scams.

Giveaway scams involve the impersonation of reputable entities such as exchanges, public figures, or projects to solicit cryptocurrency transfers under the false promise of propor-

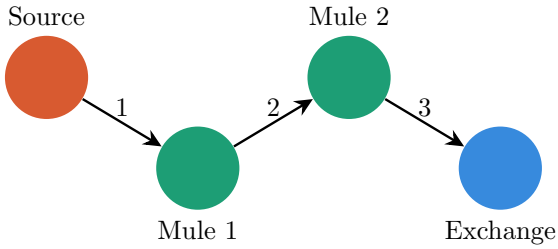


Figure 3.6.: Money laundering (layering). Funds hop through intermediary nodes rapidly before cashing out (edges 1–3). The chain length k varies.

tional returns (Hornuf et al., 2025). They may produce a burst of inbound transfers from victims to an attacker address, followed by little further activity. Real-CATS contains this exact label, but only 27 active addresses in the release used here, below the pre-specified threshold of 100. It is therefore described but excluded from RQ3 and RQ4 inference.

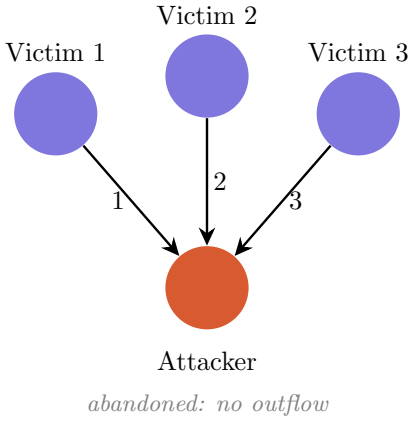


Figure 3.7.: Giveaway scam. A single inbound burst from many victims (edges 1–3). The illustrated feature is the absence of any outgoing edge from the hub within the window.

Impersonation Scam.

Impersonation scams use the identity of a trusted person, service, or project to induce a victim to transfer assets or approve an interaction. They overlap with the social mechanism of phishing, but Real-CATS records Impersonation Scam as a separate exact label. Possible on-chain patterns include repeated victim-to-scammer transfers and consolidation through common beneficiary addresses. The experiment does not assume that either pattern must be present.

Contract Exploit Scam and Hack Scam.

A contract exploit abuses a defect in deployed code, whereas the broader Hack Scam label covers compromises whose entry point need not be a smart-contract flaw (Shi et al., 2025). Both can be followed by rapid sweeps, relay transfers, or distribution across token and internal-transaction layers. The two labels are not merged in this thesis, because doing so would erase the distinction supplied by the dataset before RQ3 tests whether their fingerprints can be separated.

Blackmail Scam and Fake Returns Scam.

Blackmail scams demand payment through coercion, while fake-return schemes solicit funds by promising a payment or investment return that is not delivered. Either may produce repeated inbound transfers, but neither label prescribes one transaction topology. Their temporal fingerprints are therefore learned from the recorded histories rather than defined from a hand-written temporal template.

NFT Airdrop Scam.

NFT airdrop scams use unsolicited or fraudulent token distributions to attract a victim to a malicious interaction. Ethereum Request for Comments (ERC) token standards define how these assets are represented and transferred. The transactions can be concentrated in the ERC-721 and ERC-1155 layers and can appear as distribution stars followed by later victim interactions. Keeping the Real-CATS layers separate allows RQ3 to retain this layer-specific information.

Pigbutchering Scam.

Pigbutchering scams combine social engineering with a longer confidence-building phase before repeated transfers are solicited. This can yield a transaction history that is temporally more extended than a single theft event, but the duration is not fixed in advance. RQ4 therefore selects windows from the data and tests whether using a separate window for each category improves classification over one shared window.

The descriptions above motivate why temporal structure may be informative. They do not define the fingerprints or their time windows. Table 3.2 therefore summarizes only the nine exact labels used for inference and states the network pattern that each label could plausibly expose. These are hypotheses for interpretation, not rules used to construct labels or features. The broader Ponzi, pump-and-dump, money-laundering, and giveaway examples remain part of the domain context, but no result in Chapter 5 assigns them an empirical δ.

Table 3.2.: The nine exact Real-CATS categories evaluated in RQ3 and RQ4 and their hypothesized temporal-network interpretation.

Exact category	**Reported mechanism**	**Possible temporal-network signal**
Blackmail Scam	Coercive payment demand	One or repeated inbound payments, with no fixed topology assumed
Contract Exploit Scam	Abuse of deployed contract code	Contract-centred sweeps and multi-layer outflows
Fake Returns Scam	False promise of repayment or investment return	Repeated deposits and selective outward transfers
Hack Scam	Compromise not limited to a contract flaw	Sweeps, relays, or rapid distribution after compromise
Impersonation Scam	Misrepresentation as a trusted entity	Victim fan-in and consolidation at shared beneficiaries
NFT Airdrop Scam	Fraudulent token distribution or claim	Token-layer distribution stars and follow-up interactions
Phishing Scam	Deceptive transfer or malicious approval	Victim fan-in followed by attacker-controlled sweeps
Pigbutchering Scam	Trust-building followed by repeated solicitation	Multi-stage deposits over an extended interaction
Rug Pull Scam	Withdrawal of project or pool liquidity	Accumulation followed by concentrated contract outflow

4. Method

4.1. System overview

This chapter describes how the detection approach is put into practice. The method is implemented as a single program that turns raw Ethereum transaction records into malicious-address scores and category-level comparisons. The program is not used interactively. It runs a fixed sequence of steps, always in the same order, so that the same data and the same settings always lead to the same result.

The program runs in two phases. A *preparation* phase reads the raw data once, decides which addresses take part in the study, and fixes how they are divided into a development set and a test set. An *execution* phase then carries out the actual detection on top of these prepared data. Keeping the two apart means that the data and the evaluation setup are settled before any model is trained, so the reported results cannot be shaped, even by accident, by later changes to the data or the settings. The execution phase is checkpointed as well. After an expensive subgraph count has been committed successfully, an interrupted run verifies and reuses it instead of starting that count again.

Overall the program moves from raw transactions to results in eight steps, shown in Figure 4.1. The first two belong to the preparation phase, the remaining six to the execution phase.

1. **Loading the data.** The program reads the transaction records and the address labels of the chosen dataset. For the main dataset the transaction table is very large, with several hundred million transfers, so it is read in small batches instead of being held in memory all at once. The datasets are described in Section 4.2.

2. **Preparing the data.** Not every labelled address can be used. The program measures how active each address is and keeps only those with enough transactions to form a three-edge candidate pattern. It also sets aside a small number of extremely active hub addresses, so that a few exchange-sized accounts cannot dominate the analysis. The remaining addresses are split into a development set, used for tuning, and a separate test set, used only for the final measurement. For each address, the program stores just the transactions that involve it and writes these tables to disk, so that every later step works on the same data.

3. **Building the temporal graph.** The prepared transactions are loaded into a temporal graph, in which every address is a node and every transfer is a directed edge that carries the time at which it happened. Repeated transfers between the same two addresses are kept separately, because their number and their order define the candidate patterns. This step is described in Section 4.3.

4. **Counting the candidate subgraphs.** The program first checks the counter against synthetic validation templates. It then counts how often each address participates in each retained local role. Because an occurrence only counts when its transfers fall

within the selected time span, the counting is repeated for a range of time windows, from five minutes up to 30 days. This step is described in Section 4.4. Each completed count is stored as an immutable checkpoint before the next one begins.

5. **Building the fingerprints.** The local motif-role counts of an address form its fingerprint, a short list of numbers that records how often the address occupied each retained role. From the same transactions the program also computes a set of simpler behavioural features, such as how much value the address sent and received, how many partners it had, and how its activity was spread over time. This produces three views of every address: the simpler features on their own, the motif fingerprint on its own, and the two together.

6. **Running the detection.** For each of the three views, the program trains a classifier to tell malicious and benign addresses apart. It uses the development set to choose the time window that works best and then measures how well the classifier performs on the untouched test set. For RQ3 and RQ4 the same idea is used, except that the classifier sorts criminal addresses into their behaviour category rather than making a malicious-or-benign decision. The features and models are described in Section 4.5.

7. **Analysing the results.** The program then answers the research questions. It compares the three views to see whether the motif fingerprint improves detection over the simpler features (RQ1) and how the time window affects its usefulness for detection (RQ2). It then checks whether different categories of malicious behaviour have distinct fingerprints (RQ3), and whether selecting a separate window for each category improves classification over one shared window (RQ4). Each comparison is supported by statistical tests, so that a difference is not mistaken for chance, and the whole run is repeated under several orderings of transactions that share the same timestamp, to confirm that the outcome is stable. The evaluation is described in Section 4.6.

8. **Visualising the results.** Finally, the program writes the tables and figures used in the Results chapter, including the comparison of the three views, the performance curve across the different time windows, and for RQ3 and RQ4, the category fingerprints and the comparison between category-specific and shared windows.

The first two research questions run on the main binary dataset and the last two on the category-labelled dataset. The following sections describe the most important of these steps in more detail, starting with the data.

4.2. Data acquisition

Studying coordinated malicious behaviour on-chain needs two things at once: transaction data, and reliable labels that say which addresses are malicious. This combination is scarce. Transactions are public, but the intent behind an address can only be established off-chain, through abuse reports, exchange records, or investigations, and such labels are rare and unevenly spread across the network (Shi et al., 2025). To work around this,

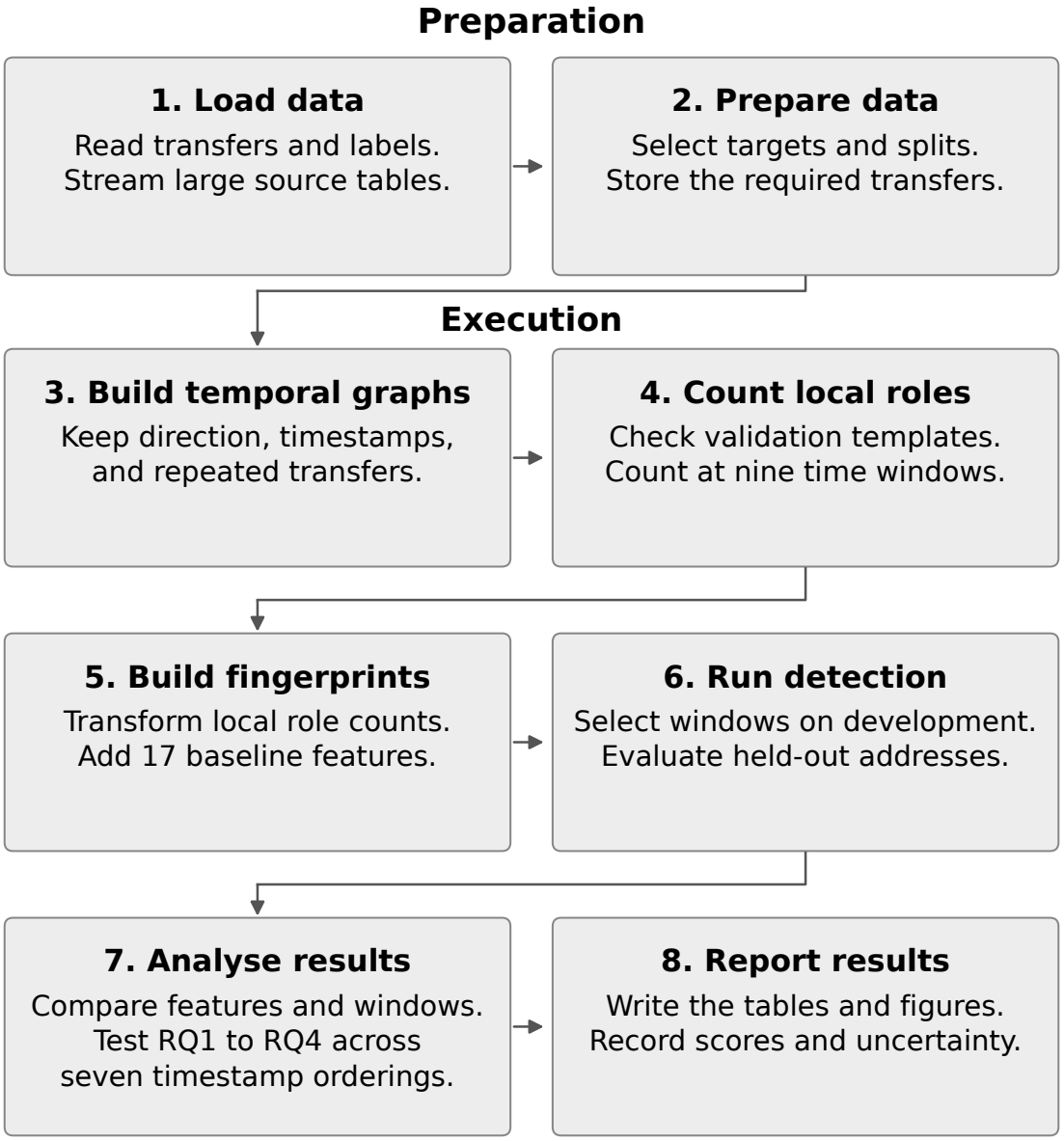


Figure 4.1.: The eight steps of the pipeline. The first two make up the preparation phase. The remaining six cover graph construction, fingerprint extraction, detection, analysis, and reporting in the execution phase.

the reported evaluation uses a main Ethereum dataset and the category-labelled Real-CATS dataset. Separate synthetic templates check the correctness of subgraph counting. The pipeline also supports BLTE as an optional secondary check, but no BLTE result is used to answer the research questions in this thesis. The origin and prior use of the two published datasets, BLTE and Real-CATS, were already covered in Section 2.3. Here each real-world dataset is described by its source, its contents, and its suitability for this thesis. Validation templates and the distinction between predictive tests and network null models are described separately below.

4.2.1. Real-world datasets

Hugging Face (HF) Ethereum Fraud Dataset.

Source. The dataset is published on the Hugging Face Hub (fesevu, 2025). This thesis uses its full-history release, which pairs a complete transaction table with a set of labelled addresses, rather than one of the smaller monthly extracts.

Contents. The dataset has two parts. The first is a transaction table of 380,566,661 transfers recorded between August 2015 and August 2025. Each transfer carries:

- the sending and receiving address, as hashed identifiers,
- the timestamp of the transfer,
- the transferred value,
- the transaction hash and block number,
- a flag marking contract-creation transactions, and
- the transaction fee.

The second part is a label table of 114,739 addresses, of which 60,086 are benign and 54,653 fraudulent. Each labelled address also states whether it is a contract or an ordinary account, and carries a short text description of the reported activity.

Suitability. This is the main dataset of the thesis, and it carries the evaluation for RQ1 and RQ2. It is the only dataset used here that combines full timestamped transaction histories with a large and roughly balanced set of malicious and benign labels. The timestamps matter because temporal motifs depend on the exact timing and order of transfers, and the size of the label set is large enough to draw more than one cohort of addresses while still keeping an untouched test set aside.

BLTE.

Source. BLTE was introduced by Al-Emari et al. (2021) and is built on the Ethereum Classic chain. Its origin and the results reported on it by earlier work were described in Section 2.3. Here it is used as obtained from the public release.

Contents. The dataset contains 71,250 transactions from October 2017 to December 2019, labelled one transaction at a time as normal or abnormal. There are 57,000 normal and

14,250 abnormal transactions, so about one transaction in five is abnormal. Each record provides:

- the transaction hash and nonce,
- the sending and receiving address,
- the transferred value, the gas, and the gas price,
- the block timestamp, block number, and block hash, and
- four labelling fields that mark whether the sender or the receiver is a known scam, and of which category.

A set of pre-computed per-address features is distributed alongside the raw transactions, but this thesis does not use them, for the reason given below.

Suitability. BLTE can be used as an optional secondary check of whether the main finding also appears in another Ethereum-based dataset. It is less directly comparable with the main dataset because its labels apply to transactions rather than addresses and because only 169 of its 73,034 active addresses are marked as scams. A BLTE run is therefore kept separate from the main evaluation and uses class weighting to account for this imbalance. The pre-computed features are left aside so that the same behavioural features can be computed as for the other datasets. No completed BLTE result is included in the Results chapter, and BLTE is not needed to answer any of the four research questions.

Real-CATS.

Source. Real-CATS was introduced by Shi et al. (2025). Its design and the sources of its labels were described in Section 2.3. This thesis uses its Ethereum portion together with the detailed transaction records released alongside it.

Contents. The Ethereum portion covers 12,572 criminal and 16,019 benign addresses. For each address the dataset provides its transaction history, split into five separate layers: ordinary transfers, internal transfers, and ERC-20, ERC-721, and ERC-1155 token transfers. What sets Real-CATS apart from the other datasets is that its criminal addresses carry a specific category, rather than a single malicious flag. The categories are:

- Phishing Scam and Impersonation Scam,
- Contract Exploit Scam and Hack Scam,
- Blackmail Scam and Fake Returns Scam,
- Rug Pull Scam, NFT Airdrop Scam, and Pigbutchering Scam.

Suitability. Real-CATS is the dataset used for RQ3 and RQ4. Its detailed categories are what make both questions possible, because they allow the motif fingerprints of different reported malicious behaviours to be compared directly, rather than only a single malicious-versus-benign split. The labels do not state whether an address belongs to a coordinated campaign, so the experiment tests category-level fingerprint differences rather than independently verified coordination. The labels are also independent of the main

dataset, since they come from public abuse reports instead of the process behind the HuggingFace release. The categories are very unevenly filled, with phishing and impersonation together accounting for most of the criminal addresses, so RQ3 and RQ4 are limited to the categories that have enough addresses to support a fair comparison and treat the smaller ones as descriptive only. The catch-all category *Other* is excluded as well, even though it is large enough to qualify, because it collects unrelated reports and therefore does not describe one kind of behaviour.

Table 4.1.: The real-world datasets and their roles. The malicious share is computed over addresses. Synthetic validation templates are described separately in Section 4.2.2.

Property	**HuggingFace**	**BLTE**	**Real-CATS**
Chain	Ethereum	Ethereum Classic	Ethereum
Transactions	380.6 million	71,250	per-address histories
Label level	address	transaction	address
Address coverage	114,739 labelled	73,034 active	28,591 labelled
Malicious share	47.6 %	0.23 %	44.0 %
Date range	2015–2025	2017–2019	per release
Category analysis	no	no	9 retained categories
Role	RQ1, RQ2	optional check	RQ3, RQ4 and binary check

4.2.2. Validation templates and null models

Synthetic validation templates.

The validation data is generated in code. Each template contains exactly three directed transfers on two or three nodes, with known directions and timestamps and no additional background edges. The templates systematically cover the patterns accepted by the counter. They do not simulate complete fraud campaigns or provide malicious-address labels. A three-node structure may resemble one part of a plausible scam, such as collecting and forwarding funds, but it remains a candidate motif before any statistical comparison.

Raphtory returns an ordered vector of 40 local motif-role counts for each address. A position means one component of that vector, not one of 40 global patterns. The templates check all 24 star-centre roles, eight two-node roles, and eight triangle roles. The difference from the 36 global patterns of Paranjape et al. (2017) is explained in Section 4.4.1. The templates also check transfers exactly at a window boundary and transfers with equal timestamps. Their purpose is to confirm that subgraph counting recognises a known two- or three-node structure and assigns the expected local role counts. They are never used to measure detection performance or compared with real-world datasets as examples of fraud.

Network null models.

A validation template has a known answer for the counter. A network null model serves a different purpose: it generates random reference networks against which the observed

frequency of a candidate pattern can be tested (Milo et al., 2002). For example, shuffling timestamps while retaining the observed directed transfers would test their temporal arrangement conditional on that fixed structure. The choice of preserved properties determines the claim that such a comparison can support.

The completed experiments do not include a random-network null model, so no individual candidate is claimed to be statistically overrepresented. The permutation and bootstrap procedures in Section 4.6 instead assess predictive differences and category separability. The alternative orderings of equal-timestamp transfers are sensitivity checks, not random-network references.

4.3. Temporal graph construction

The prepared transactions are turned into a temporal graph on which candidate subgraphs are later counted. This section describes the graph model, how the graph is actually built from each dataset, and the simple features that are read straight from it and later serve as a comparison point for the fingerprints.

4.3.1. Graph model

Each address is represented as a node and each transfer as a directed edge that carries the time at which it happened. The result is a temporal directed multigraph

$$\mathcal{G} = (V, E, \tau, w),$$

where V is the set of addresses, E the set of transfers, $\tau(e)$ the timestamp of transfer e, and $w(e)$ its transferred value. Two properties are kept on purpose. Parallel transfers between the same pair of addresses are not merged, because how many there are and in which order they occur is exactly what the candidate patterns are built from. Edge direction is kept for the same reason, since an address that only receives, one that only sends, and one that does both leave different local role counts behind.

4.3.2. Building the graph from the data

HuggingFace and Real-CATS are stored in very different shapes, so the graph is assembled differently for each, but always without holding more data in memory than needed.

For HuggingFace the transaction table is far too large to load as a whole, with more than 380 million transfers. It is therefore read in small batches. The first pass measures how active each labelled address is and keeps every transfer that touches a selected target. These incident transfers define the baseline and the set containing each target and all of its direct partners. A second full scan keeps every published transfer whose two endpoints both belong to that set. The resulting released-neighbourhood closure is used for subgraph counting.

Every triangle involving a selected target consists of two target-to-partner edges and one edge between those partners. The first two edges place both partners in the retained

node set, and the second scan then keeps their connecting edge. Triangle counts are therefore complete relative to the published release. They are not claimed to be complete for Ethereum activity absent from the release. Baseline features remain restricted to target-incident transfers so that adding the closure does not give the comparison model extra network scope.

Real-CATS is stored as one transaction history per address, and the same transfer can appear in the histories of several addresses. These repeated copies are removed once, using a disk-backed index so that the de-duplication does not depend on everything fitting in memory. Each history is also split into five layers, namely ordinary transfers, internal transfers, and ERC-20, ERC-721 and ERC-1155 token transfers. Every layer becomes its own set of edges, and candidate subgraphs are counted within each layer. This preserves the distinction between transaction types, but excludes candidate sequences whose edges cross between layers.

4.3.3. The non-motif baseline

RQ1 asks whether motif fingerprints add anything beyond ordinary address behaviour, so it needs a description of each address that does not use motifs at all. This baseline is computed from the same transfers and the same addresses as the motif fingerprints. Table 4.2 lists the resulting 17 features, grouped into how active an address is, how much value it moves, the structure of its immediate neighbourhood, and how its activity is spread over time.

Table 4.2.: The non-motif baseline features used for RQ1.

Feature group	**Features**
Activity	Number of transfers sent, received, and in total
Transferred value	Outgoing and incoming sum, mean, and sample standard deviation, plus the total value moved
Neighbourhood structure	Number of distinct outgoing neighbours, number of distinct incoming neighbours, and reciprocity, the share of an address's neighbours that it both sends to and receives from
Relative timing	Active span and the mean, median, and sample standard deviation of the gaps between successive transfers that touch the address

A few choices keep this baseline honest. Absolute first and last transaction times are deliberately left out, because they mostly reveal when the data was collected or how old a campaign is rather than how an address behaves, and a model could otherwise separate the classes by collection window instead of by behaviour. Measures that describe an address's position in the wider network are also left out, because computing them reliably would need more of the surrounding graph than is kept here. Where a value cannot be computed, for instance a measure of variation for an address with too few transactions, it is set to zero.

4.4. Temporal motif extraction

The baseline describes an address by totals, averages, and timing summaries. It does not retain the full order of transfers between particular partners. Two addresses can have the same number of transfers, the same partners, and the same active span, yet behave very differently: one might receive money from many senders within minutes and pass it straight on, while the other collects the same money slowly over months. Candidate temporal patterns capture these differences. Recall from Section 3.4 that a candidate is a short, time-ordered sequence of transfers that all fall within a window of length δ. Choosing a focal address determines its local role in an occurrence. Stacking the address's local role counts into a vector gives its motif fingerprint (Equation 3.1). This section describes how those fingerprints are counted and turned into features.

4.4.1. Subgraph counting and retained local roles

The counting itself is done by Raphtory, whose `local_temporal_three_node_motifs` routine is written in Rust and processes addresses in parallel. For every address it returns a *local* count for each motif role, that is, the number of candidate occurrences in which the address plays that particular part. A star pattern is counted only for the address at its centre, and a two-node pattern for both of the addresses involved.

Raphtory reports 40 such roles per address: 24 star roles, 8 two-node roles, and 8 triangle roles. The star roles comprise three classes of eight in-and-out direction patterns each. The four global two-node patterns each have two endpoint roles, giving eight local two-node counts. This accounts for the difference between 36 global patterns and 40 local role features. The output is not a catalogue of 36 global motifs with four additional patterns. The global patterns are shown in Figure 3.1. The final HuggingFace experiment retains all 40 roles because its released-neighbourhood closure supports triangle counting. It also evaluates the 32 star and two-node roles separately to measure whether triangles add signal. Real-CATS retains those 32 roles per layer because its collected histories cannot guarantee the edge between two partners. An address that takes part in no retained candidate pattern keeps a fingerprint of all zeros rather than being dropped from the study.

The retained roles fall into a few structural classes, each with a straightforward reading in a transaction network. Table 4.3 lists them together with the kind of coordinated behaviour they could represent. These interpretations do not label an occurrence as fraudulent. In every example, the timing condition is $t_3 - t_1 \leq \delta$, with δ drawn from the five-minute to 30-day grid in Section 4.4.4. There is no separate threshold for *rapid* activity, and an occurrence admitted at 30 days need not be rapid in an everyday sense.

The terminology in Table 3.1 applies throughout this chapter. Each feature column describes a local role, each class groups related roles, and the fingerprint contains the complete set of retained counts for one address.

Table 4.3.: Retained structural motif classes (after Paranjape et al. (2017)) and their behavioural interpretation in Ethereum transaction networks. Each class covers several δ-temporal roles that differ in the direction and order of the three transfers.

Class	Ordered transfers	Possible interpretation
Two-node, same direction	$A \to B,\ A \to B,\ A \to B$	Repeated transfers, which may indicate instalment-based extraction
Two-node, mixed direction	$A \to B,\ B \to A,\ A \to B$	Back-and-forth transfers within δ, as in wash-trading or fee manipulation
Star, all incoming	$B \to A,\ C \to A,\ C \to A$	Funds consolidating at a single hub, as in money-mule collection
Star, all outgoing	$A \to B,\ A \to C,\ A \to C$	Mass distribution from one source, as in airdrop farming or Ponzi payouts
Star, mixed	$A \to B,\ C \to A,\ A \to C$	A relay node forwarding funds, as in a layering intermediary
Triangle	$A \to B,\ B \to C,\ C \to A$	A closed transfer sequence, shown here as a cycle. Other triangle roles have different directions.

4.4.2. Ordering transfers with equal timestamps

Because motifs are ordered sequences, the transfers have to be put in a definite order before they can be counted. Time supplies most of that order, but many transfers carry the same timestamp because they sit in the same block, and time alone cannot separate them. When the data records where a transfer sits within its block, that finer order is used. The HuggingFace data does not record this, so its transfers that share a timestamp have no fixed order. To make sure the results do not rest on an arbitrary choice, the whole analysis is run under seven versions of the order. All orderings preserve the order of transfers with distinct timestamps. They differ only in the order assigned to transfers sharing the same timestamp: one fixed order, its reversal within each timestamp group, and five random shuffles within those groups. These differences can change which candidate temporal pattern a sequence matches and which local roles are counted. Section 4.6 reports whether the conclusions hold under all seven. The order is computed without loading the whole table into memory, and the sorted transfers are passed to Raphtory in large batches.

4.4.3. Fingerprint representations

The retained counts are stored under stable names such as `star_pre_iii`, `two_node_ioi`, and `triangle_1`. For RQ1 and RQ2 each count c is replaced by $\log(1 + c)$. Some highly active addresses have much larger counts than most targets. On the raw scale these values

would dominate the model. Taking the logarithm compresses this range, and adding one first means that a count of zero stays zero. The final HuggingFace detection step uses all 40 transformed values, either on their own or joined to the baseline of Section 4.3. A parallel 32-role representation and a triangles-only representation support the ablation analysis. How the fingerprint and baseline are combined into the RQ1 feature sets is described in Section 4.5.

The representation comparisons also use *motif volume*, defined here as the sum of an address's retained local role counts. This is a local summary, not the number of distinct global pattern types.

RQ3 and RQ4 keep the five Real-CATS layers separate. Because the layers differ greatly in how many transfers they contain, comparing raw counts would let the busiest layer dominate. Instead, within each layer the counts are turned into proportions by dividing each count by the layer total, and the square root of each proportion is taken. This is known as a Hellinger transform (Legendre and Gallagher, 2001):

$$h^{\delta}_{v,j,l} = \sqrt{\frac{c^{\delta}_{v,j,l}}{\sum_{k=1}^{32} c^{\delta}_{v,k,l}}},$$

where $c^{\delta}_{v,j,l}$ is the count of local role j for address v in layer l at window δ. The expression applies when the layer's total role count is positive. Using proportions removes the total count, so only the mix of local roles remains, and the square root keeps a few large proportions from dominating, much as the logarithm does for the counts. A layer with no retained candidate occurrences is kept as all zeros, including a layer that has transfers but no complete three-edge pattern. The same counts in log form, as used for RQ1 and RQ2, are also kept as a check. Concatenating the five layers gives 160 local role features for each Real-CATS address, with 32 components per layer.

For every window the pipeline additionally records whether each address has any nonzero count at all. This coverage measure shows how the availability of fingerprints changes with the window. Reporting coverage alongside performance helps assess their relationship, but does not isolate whether a score increase comes from lower sparsity or a more informative time scale.

4.4.4. Temporal-window sweep

The counting is repeated over a fixed grid of nine windows,

$$\delta \in \{300,\ 1800,\ 3600,\ 21600,\ 86400,\ 259200,\ 604800,\ 1209600,\ 2592000\} \text{ seconds},$$

which correspond to five minutes, 30 minutes, one hour, six hours, one day, three days, one week, two weeks, and 30 days. All windows use the same addresses and the same train and test split, and the window is chosen only on the development data, as described in Section 4.6. RQ2 studies how detection changes across this grid, and RQ4 asks whether separate windows for each category improve classification over one shared window. A

selected window is the maximum span allowed for a three-edge candidate motif. It is not an estimate of the total duration of a fraud campaign.

4.5. Detection methods

The previous two sections produced two descriptions of every address: the non-motif baseline of Section 4.3 and the motif fingerprint of Section 4.4. This section describes how those descriptions are turned into a detector and which models are used. The binary detection for RQ1 and RQ2 is kept apart from the category detection for RQ3 and RQ4, because the two groups ask different questions and use different datasets.

4.5.1. The three feature views

RQ1 asks whether the motif fingerprint adds anything beyond ordinary address behaviour. The cleanest way to answer this is a controlled comparison, in which only the features change and everything else stays fixed. Three views of each address are therefore built from the two descriptions above:

- the *baseline* view, the 17 non-motif features of Table 4.2,
- the *motifs* view, the log-transformed local motif-role counts, with 32 roles in two HuggingFace samples and all 40 roles in the sample that includes triangles, and
- the *combined* view, the two simply placed side by side.

As described in Section 4.6, RQ1 and RQ2 use three non-overlapping HuggingFace samples. Two samples use the 32 star and two-node roles. The remaining sample has the released-neighbourhood closure needed for all 40 roles and evaluates the 32-role view and the eight triangle roles as separate ablations. All three views use exactly the same addresses and the same train and test split, and no address is dropped just because its fingerprint is all zeros. Because the data is held constant, this comparison evaluates the predictive value added by the fingerprint representation. The main contrast is 17 baseline features against those same 17 features plus 32 or 40 local motif-role features. Each role combines structure, direction, focal-node position, and temporal order. The experiment therefore does not isolate the contribution of temporal order alone. That would require a matching representation with the ordering removed.

RQ2 then asks how the time window affects the usefulness of the fingerprints. For this the combined view is evaluated at each of the nine windows from Section 4.4. The baseline does not depend on the window, so it stays the same throughout and serves as a fixed reference line.

4.5.2. Classifiers for RQ1 and RQ2

The detector for the binary questions is a random forest, an ensemble of decision trees that each vote on whether an address is malicious. A random forest suits this task because it can pick up interactions between different motif roles without any manual feature scaling, and because it is a strong, well-understood default rather than a model tuned hard for the

highest possible score. Its settings are fixed in advance, with 100 trees, a limited depth, balanced class weights, and a fixed random seed, and they are not adjusted on the test data. The forest is trained on the development addresses and then scores each address in the test set with a probability of being malicious. The time window is chosen on the development data by cross-validation (CV), using the rule described in Section 4.6.

As a robustness check, an Extreme Gradient Boosting (XGBoost) model is also fitted, on the combined view at the chosen window. If it agrees with the random forest, that strengthens confidence in the result, but the RQ1 decision always rests on the random forest, which is fixed beforehand. The models use the allocated central processing unit (CPU) threads. Table 4.4 summarises all three models.

Table 4.4.: The three models used in the detection experiments.

Task	**Model**	**Fixed settings**
RQ1 and RQ2 (main)	Random forest	100 trees, depth 10, balanced class weights, a fixed seed, allocated CPU threads
RQ1 (robustness)	Gradient-boosted trees (XGBoost)	100 rounds, depth 6, learning rate 0.1, positive-class weight from the training split, a fixed seed
RQ3 and RQ4	Multinomial logistic regression	Standardised features, balanced class weights, up to 2,000 iterations, a fixed seed

4.5.3. Classifier for RQ3 and RQ4

RQ3 and RQ4 do not separate malicious from benign addresses. They sort criminal addresses into their behaviour category, so the model has one class per category rather than two. Both questions are answered from the same model on the same data, and they are therefore always prepared and run together as one analysis. Only categories with at least 100 addresses take part, because the categories are very unevenly filled, as explained in Section 4.2, and categories are never merged into larger groups.

The input here is the per-layer fingerprint of Section 4.4, expressed as local role proportions, which captures the mix of roles an address occupies in each transaction layer. The model is a logistic regression in its multi-class form. Its inputs are first put on a common scale, and this scaling is learned only from the training data so that the test data cannot influence it. A logistic regression is chosen deliberately, because it is a simple model with few parameters. This tests whether the motif proportions alone can separate the categories, without a more complex model that might overfit the smaller categories.

RQ3 evaluates category separability using motif fingerprints alone. Class shares and a permutation reference provide context for this separation. There is no comparison with the conventional behavioural features for category classification, so RQ3 does not test whether fingerprints outperform those features.

Two kinds of window are selected on the development data, one for each question. A single global window is chosen to separate all categories as well as possible at once, and this is the window RQ3 uses to ask whether the categories have distinct fingerprints at all. In

addition, a separate window is chosen for each category, the one at which that category is recognised best. Whether these category-specific windows actually beat the single global window is what RQ4 tests. Both kinds of window are chosen on the development data only. The final model is refitted on all development addresses and gives each address in the test set one probability per category.

4.5.4. Handling class imbalance

The classes are uneven to different degrees across the experiments, only slightly for the main dataset and strongly for the Real-CATS categories. The same applies if the optional BLTE run is performed. This is handled in a single way, by weighting each class in inverse proportion to how often it appears, so that a rare class counts for as much as a common one during training. The random forest and the logistic regression use balanced class weights, and the gradient-boosted model uses the matching positive-class weight from its training split. No artificial examples are added, and the number of addresses in each class is left unchanged.

4.6. Evaluation

4.6.1. Scope

The evaluation measures how well an address can be classified after the fact, from its full recorded history. It does not estimate how early an address could have been caught, and it makes no causal claim about how a fraud was carried out. As described in Section 4.1, the data and every setting are fixed in a preparation step before any model is trained, so nothing in the evaluation can be tuned on the test data. The parts that enforce this, such as the recorded settings and the checks that the code and data have not changed, are described in Section 4.7.

4.6.2. Cohorts and splits

RQ1 and RQ2 are evaluated on three non-overlapping samples from the HuggingFace dataset. Each sample contains two separate groups of addresses. An address is eligible only if it takes part in at least three transactions, since fewer cannot form one of the three-edge motifs. Before the groups are drawn, the most extreme addresses are removed: within each combination of label, contract type, and activity level, the top 0.5 % most active addresses are dropped. Without this, a few exchange-scale hubs would dominate the graph on their own. The cut is made within each combination rather than across the whole dataset, because the busiest addresses are almost all benign exchange wallets, so a single global cut would remove far more benign than fraudulent addresses and quietly shift the balance between the classes. The rule is fixed before any model is trained, and every removed address is recorded.

The two groups serve different purposes. The *prevalence* cohort is a plain random sample of up to 5,000 eligible addresses, which keeps the natural mix of fraudulent and benign addresses and checks that the findings hold on a realistic distribution. The *controlled* cohort

is drawn from the remaining addresses, up to 250 per combination of label, contract type, and activity level, which balances the classes and holds activity and contract type roughly constant so that a difference cannot be explained by those alone. The two cohorts within a sample share no addresses and are reported separately. After both cohorts are drawn, their addresses and known campaign groups are removed from the eligible pool before the next sample is created. Repeating this procedure produces three samples without reusing addresses or known campaigns.

Splitting the addresses needs one precaution. Some fraud arrives in campaigns of many near-identical addresses, and if some of them landed in the training set while their near-twins landed in the test set, the detector would look better than it really is. Campaign groups are therefore kept together, and every split keeps a whole group on a single side. In the implementation, these groups are defined from the published address descriptions. After trimming whitespace and converting the text to lower case, descriptions beginning with *multichain hack alert*, *upbit hacker*, or *sorbet finance hack alert* are assigned to their respective shared family. Every other address receives its own group. These are description-based grouping rules, not a complete reconstruction of coordinated campaigns. Each cohort is divided about 80 to 20 into a development set and a test set with a group-aware, stratified split, and the development set is divided again into repeated five-fold splits, under different random seeds, for choosing the time window. Cohort membership is included in the stratification, while campaign assignments are made across both cohorts together. A campaign represented in both cohorts therefore remains on the same side and in the same validation fold. RQ3 and RQ4 share their own 80 to 20 split, stratified by category, again keeping only categories with at least 100 addresses and never merging them.

4.6.3. Metrics and window selection

The main measure is average precision (AP), which summarises how well the detector ranks malicious addresses above benign ones across all decision thresholds. It suits imbalanced data because it focuses on the rarer positive class. The F1 score is the harmonic mean of precision and recall. These three measures are reported as secondary measures at a fixed threshold of 0.5, which is never tuned on the test set. The area under the receiver operating characteristic curve (ROC-AUC) is reported alongside them. For RQ3 and RQ4 the matching measures are the macro-averaged and per-category average precision, the macro F1, and a confusion matrix.

The time window is chosen on the development data alone. Each window is scored by cross-validation, and the window with the highest average score is taken. If several windows share that score, the shortest of them is used, so that the choice is decided in advance rather than by chance. As a check on how firm that choice is, the shortest window whose score is still within one standard error of the best is recorded alongside it as the one-standard-error (1-SE) choice, which shows whether a shorter window would have done essentially as well. The test set plays no part in either.

4.6.4. Statistical tests

For RQ1 the effect is the difference in test-set average precision between the combined view and the baseline. Two procedures support it. The first is a paired permutation test. For each campaign group, a random choice determines whether the baseline and combined prediction scores are exchanged for every address in that group. Address labels, campaign membership, and the train and test assignments stay fixed. The models are not retrained. The difference in AP is recomputed for 10,000 such exchanges, giving a one-sided p-value under the null that the two sets of prediction scores are exchangeable within groups.

The second procedure is a bootstrap with 10,000 repetitions. Groups are sampled with replacement separately within the benign and malicious classes. Each class retains its original number of sampled groups. Every sampled group contributes all of its addresses, including repeated copies if the group is drawn more than once. The paired predictions are kept together and the AP difference is recomputed, giving a 95 % confidence interval (CI). The groups are the description-based families and singleton addresses defined above.

Groups can contain different numbers of addresses. Each group has the same chance of being sampled within its class, but a larger sampled group contributes more addresses to AP. The total number of addresses and the class proportions can therefore vary between bootstrap repetitions. The score-swap test, in contrast, always uses the original test addresses. Both procedures preserve dependence within the known groups, while the reported metric remains an address-level measure. In the binary Real-CATS check, groups are individual addresses because campaign identifiers are unavailable.

For RQ2 the chosen window is compared with each of the other windows using the same kind of bootstrap. Because many windows are compared at once, the resulting p-values are adjusted for multiple testing with the Benjamini–Hochberg procedure (BH). The development curve and fingerprint coverage describe how usefulness changes with δ. The pairwise tests additionally check whether the selected window is distinguishable from the alternatives. A unique best window within the tested grid is supported only if it beats every alternative after correction.

RQ3 asks whether the fingerprints contain category information. The held-out category labels are shuffled 10,000 times while the fitted model and its prediction probabilities stay fixed. Each shuffle preserves the class sizes. The resulting macro AP values form a reference for no association between the predictions and the category labels. Failure to reject this null would mean insufficient evidence of category separation in this test. It would not prove that the fingerprints contain no category information.

RQ4 asks whether the categories are better served by their own windows than by one shared window. The two settings are compared on the test set in two ways. The first is an overall comparison: the model using each category's own window is set against the model using the single global window, and the difference in macro-averaged average precision is measured with a paired bootstrap. The second looks at each category on its own, comparing its own window with the global one, and because this is repeated for every category the results are adjusted for multiple testing. The number of distinct windows the

categories select is reported alongside, since category-specific windows can only help if the categories do not all choose the same one.

4.6.5. Robustness to event ordering

Every result is produced under each of the seven orderings of Section 4.4. An ordering-dependent conclusion, such as an RQ1 improvement or an RQ2 best window, is accepted only when its direction and its decision agree across all seven orderings. This shows that a conclusion does not depend on how transfers sharing a timestamp are ordered. It does not recover the real order of those transfers, which the data does not record in the first place.

4.7. Implementation details

This section records the tools the pipeline is built on, how the code is organised, and the measures that make a run reproducible. The engineering challenges are documented in Appendix A.

4.7.1. Language and libraries

The pipeline is written in Python, with the temporal motif counting carried out by Raphtory's Rust extension. Everything runs on the CPU, so no graphics card is needed, and Raphtory, DuckDB, the random forest, and XGBoost are all set up to use the thread allocation recorded for the run. Native thread limits are set before the numerical libraries are imported, so two coordinated workers cannot each claim every core. Development and validation used Python 3.13 in an isolated virtual environment. The exact interpreter and package versions of each run are recorded in its protocol, and that record, rather than the environment described here, is what counts for the final numbers. Table 4.5 lists the main libraries and their role.

NetworkX and the HuggingFace `datasets` client are used only by the exploratory application and by the data download, not by the locked pipeline. SQLite, which comes with Python, provides the index used to de-duplicate the Real-CATS histories.

4.7.2. Project structure and entry points

The repository keeps the exploratory and the evidence-producing code apart. The interactive command-line interface (CLI) in `main.py` is for development and inspection, while the thesis results come only from `scripts/run_results.py`, whose `prepare` and `execute` subcommands call the separate `src/experiments` package. The same entry point provides read-only `preflight` and `verify` commands, execution `-resume`, and a coordinated `execute-parallel` command for running the HF and Real-CATS roots on one server. Keeping the two apart stops exploratory shortcuts, such as row limits or changed defaults, from leaking into the results. The main files are shown below, and Figure 4.2 shows how they connect across the two phases of a run.

Table 4.5.: The main libraries used by the pipeline.

Library	Version	Role in the pipeline
Raphtory	0.17.0	Stores the temporal multigraph and runs the parallel Rust routine `local_temporal_three_node_motifs`, using the local counting approach of Arnold et al. (2024). Its full 40-role output is retained for HuggingFace. The 32 star and two-node roles are retained for each Real-CATS layer.
DuckDB	1.5.4	Queries the prepared Parquet directly, computes the baseline, and sorts events for each ordering variant with bounded memory.
PyArrow	23.0.1	Streams the full HuggingFace edge source, writes the prepared Parquet files, and carries the ordered batches from DuckDB to Raphtory.
pandas	3.0.2	Holds the address-level targets, fingerprints, model matrices, predictions, and metrics. The primary HF and Real-CATS event tables are not materialised together in pandas. The much smaller BLTE table is.
NumPy	2.4.4	Provides numerical arrays and the deterministic random numbers used by the models, the resampling, and the metrics.
scikit-learn	1.8.0	Supplies the group-aware and stratified splitting, the random forest, the standardisation and logistic regression, and the metrics.
XGBoost	3.2.0	Provides the gradient-boosted robustness model for the combined RQ1 view.
matplotlib	3.10.8	Produces the figures written for the Results chapter.

```
bachelor_thesis/
main.py               CLI entry point, pipeline orchestrator
config.py             Exploratory defaults only
requirements.txt      Environment definition
requirements-results.txt Exact numerical Results environment
scripts/run_results.py  Locked Results prepare/execute CLI
src/
 experiments/
   constants.py     Locked constants and verified role catalogue
   prepare.py       Source conversion, cohorts, folds, protocol lock
   datasets.py      HuggingFace streaming and BLTE loading
   realcats.py      SQLite deduplication and layer export
   ordering.py      DuckDB external event ordering
   features.py      Baseline and fingerprint transformations
   motifs.py        Arrow-to-Raphtory loading and motif sweep
   modeling.py      Fixed estimators and development selection
   statistics.py    Metrics, permutations, bootstrap, BH correction
   splits.py        Group-aware and stratified partition construction
   rq_binary.py     RQ1/RQ2 execution and persistence
   rq_multiclass.py RQ3/RQ4 execution and persistence
   secondary.py     Conditional BLTE and synthetic validation
   synthetic_fixtures.py  Local-role and boundary validation templates
```

```
    execute.py       Verification and RQ orchestration
    execution_control.py Locks, attempts, threads, state transitions
    execution_artifacts.py Checkpoints, stages, result inventory
    coordination.py  Same-host HF/Real-CATS coordination
    verify.py        Read-only integrity and citability checks
    validation.py    Prepared-data and result-schema guards
    artifacts.py     Exclusive writes, hashes, runtime provenance
    resources.py     Read-only memory/disk projection and admission
    telemetry.py     Timing log and five-minute heartbeat
    types.py         Shared data contracts (dataclasses)
    visualization.py Chapter-ready figures
 graph_construction.py  Exploratory NetworkX graph features
 motifs/            Exploratory alternative motif backends
data/raw/            Raw datasets (gitignored)
results/experiments/ Locked run directories
```

Listing 4.1: Project layout (abbreviated).

4.7.3. Reproducibility

The study is built around a strict separation between fixing the experiment and running it, so that no choice can be influenced by the test results. All settings live in one place, `src/experiments/constants.py`, and are copied into the run's protocol when preparation finishes. The main ones are shown below.

```python
RANDOM_SEED = 42
CV_SEEDS = (42, 43, 44)
DELTAS = (300, 1800, 3600, 21600,
          86400, 259200, 604800,
          1209600, 2592000)
N_PERMUTATIONS = N_BOOTSTRAPS = 10_000
HF_COHORT_ELIGIBILITY = {
    "minimum_incident_edges": 3,
    "activity_upper_quantile": 0.995,
}
```

Listing 4.2: Selected fixed experiment settings.

When preparation finishes it writes `protocol.json`, which records the data (the source revisions and a checksum of the source, the prepared files, the exact target addresses, and the cohort and fold assignments), every model, window, ordering, resampling, and seed value, and checksums of the code and of the installed package and Python versions. A checksum is a short code computed from a file that changes whenever the file changes, so it flags any later edit. From that point the protocol and the prepared data are fixed. When a run is executed it recomputes all of these checksums and refuses to produce a single feature or metric if the code, the data, the versions, or the settings differ from what was locked. Fixed seeds make the cohort sampling, the folds, and the resampling repeatable, and the deterministic synthetic templates of Section 4.2.2 check that the subgraph counter assigns the roles it should, including right at the edge of a time window. The thesis profile

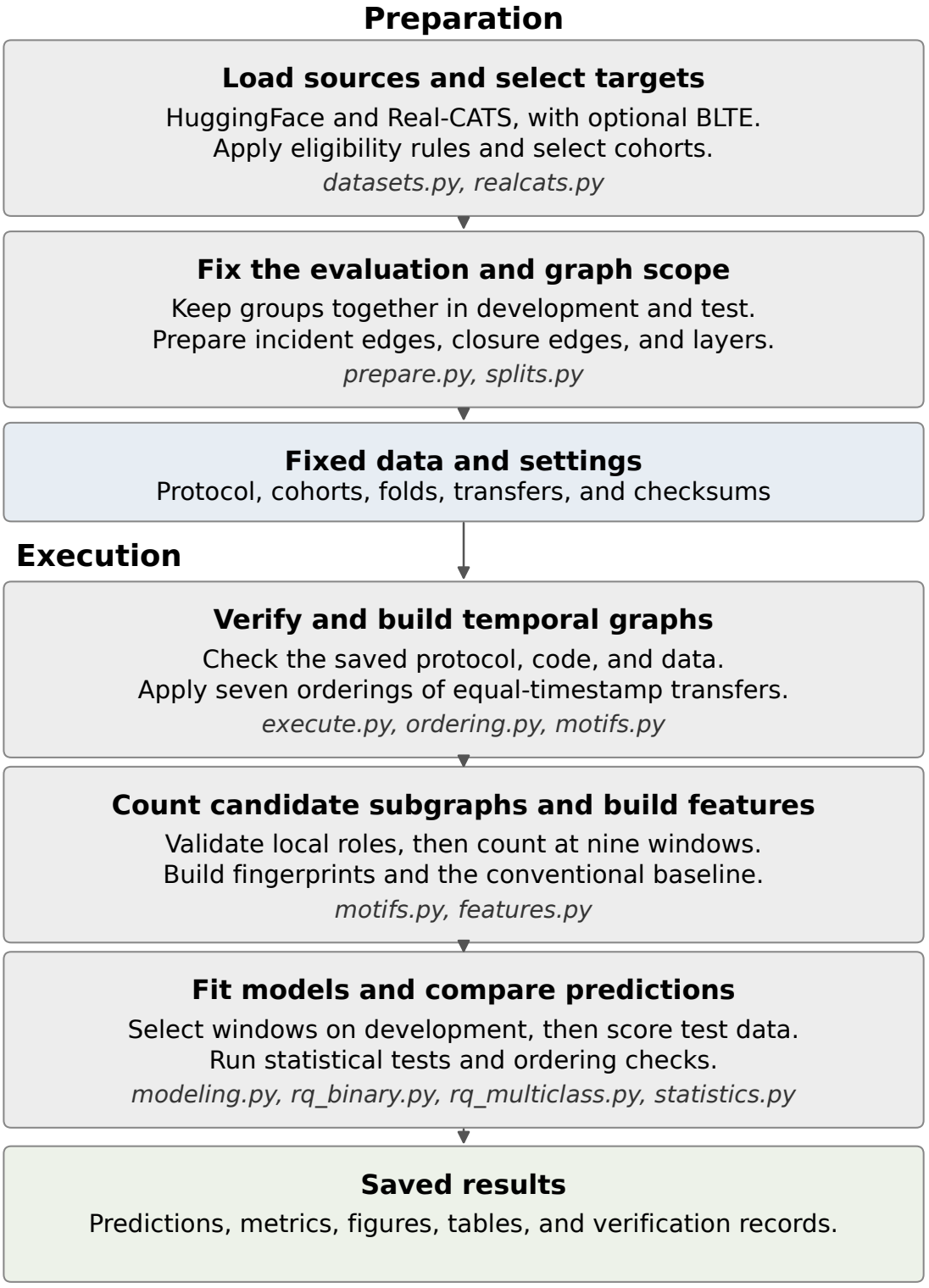


Figure 4.2.: The locked Results pipeline. The preparation phase fixes the cohorts and writes the locked artifacts, and the execution phase then computes the local role counts, trains the models, and runs the statistical tests. Italic labels name the source modules in `src/experiments`.

fixes all settings above. A separate development profile may reduce only the numbers of permutation and bootstrap repetitions and is always marked non-citable.

Before execution, a read-only preflight checks the recorded random-access memory (RAM) and disk budget without creating a lock, log record, temporary directory, or timestamp change in the run. Execution then records its operational settings in the immutable `execution_definition.json`. This second definition separates operational choices, including the thread allocation, direct or coordinated mode, coordinator identity, and concrete worker thread settings, from the analytical model parameters in the protocol. A resumed run must use the same allocation. The thread limits are set before importing NumPy, scikit-learn, or the native libraries, which prevents numerical thread pools from silently exceeding that allocation.

Exact equality is required for integer motif counts, address order, splits, selected windows, and hypothesis decisions. Independently recomputed floating probabilities and metrics are compared within a numerical tolerance because Basic Linear Algebra Subprograms (BLAS), XGBoost, and Parquet encoders need not be byte-identical under different thread allocations. Reusing an existing verified checkpoint or stage, in contrast, is byte-preserving. Portable Document Format (PDF) timestamps are removed so their hashes do not change only because of generation time.

Scientific guards run before finalisation. They reject overlapping or duplicate targets, campaign groups that cross a split, inconsistent folds, missing classes, negative motif counts, non-finite features or metrics, duplicate prediction rows, probabilities outside $[0, 1]$, and multi-class probability vectors that do not sum to one within numerical tolerance. Conflicting duplicate labels in Real-CATS are rejected, while identical duplicates are collapsed and counted in the preparation audit. All five layer files are required even when one is empty. Checkpoint content is hashed from a fixed Arrow representation with named columns, 64-bit integer counts, and addresses sorted by their 8-bit Unicode Transformation Format (UTF-8) bytes, independently of its Parquet storage encoding.

4.7.4. Result artifacts

Each run has its own directory that is never overwritten, so results from different runs cannot be mixed, and a run may only create a file that does not yet exist. Prepared edges, fingerprints, and test-set predictions are stored as Parquet, the protocol and the test summaries as JavaScript Object Notation (JSON), the chapter tables as comma-separated values (CSV), and the figures as PDF. Output files are first fully written and synchronised in a private attempt directory. Files are committed without replacement, and a multi-file stage is published by an atomic directory rename after its manifest is written last. The public `features`, `metrics`, and related directories contain hard links to verified stage payloads, preserving the filenames consumed by the Results chapter.

Preparation and execution have separate resume rules. During Real-CATS preparation, the SQLite index records each completed address and can be resumed only before the experiment settings are fixed. During execution, every attempt has its own identifier and

scratch directory. HF commits one subgraph-count checkpoint for every ordering and window. Real-CATS commits one for every layer, ordering, and window. A new attempt may reuse earlier valid commits. A completed run rejects both a fresh execution and a resume.

The integrity chain is acyclic. The locked protocol and prepared data identify the execution definition. These identify every checkpoint and stage. The result inventory hashes the committed stage manifests and public files, and the execution manifest records the inventory hash. Each inventory and manifest has a separate checksum file and does not include itself. The final `verify` command checks this chain and the scientific schemas without writing to the run. Ordinary verification distinguishes a valid non-citable development or smoke run from corruption. Final thesis acceptance additionally requires `-require-citable`.

The only appended file is `run_log.jsonl`, which stores one JSON record per line in JSON Lines (JSONL) format. Its writes are thread-safe and synchronised to disk. A native operation that runs for several minutes emits a heartbeat every five minutes with its elapsed time, allocated threads, free disk, and latest committed checkpoint. The output layout is shown below.

```
results/experiments/<run-id>/
  preparation_state.json
  protocol.json / protocol.sha256
  execution_definition.json / execution_definition.sha256
  attempts/<attempt-id>/
    started.json, completed.json or failed.json
    scratch/duckdb/
    staging/
  checkpoints/<checkpoint-id>/
    counts.parquet
    metadata.json
  stages/<stage-id>/
    payload/
    stage_manifest.json
  run_log.jsonl
  data/
  features/
  predictions/
  metrics/
  figures/
  tables/
  result_artifacts.json / result_artifacts.sha256
  execution_manifest.json / execution_manifest.sha256
```

Listing 4.3: Locked Results artifact layout.

5. Results

This chapter presents the results for the four research questions. RQ1 and RQ2 are examined in three non-overlapping HuggingFace samples. Each contains a randomly drawn prevalence cohort and a controlled cohort in which benign and malicious addresses are matched by activity. The two 32-role samples contain only star and two-node motif roles. The 40-role sample also includes triangles. RQ3 and RQ4 use the nine Real-CATS categories with enough active addresses for the analysis. Real-CATS also provides a second binary comparison for RQ1 and RQ2. BLTE was prepared as an optional dataset, but it is not part of the results reported here.

The chapter reports the measured scores, window selections, and statistical comparisons. Their interpretation and possible explanations are discussed in Chapter 6.

5.1. Evaluation Samples

The HuggingFace source contains 114,739 labelled targets and 380,566,661 transfers. Of the targets, 48,737 have no incident transfer and 26,618 have fewer than the required three transfers. A further 52 very highly active addresses are removed, leaving 39,332 eligible addresses. Addresses and campaign groups used in an earlier sample are excluded before the next sample is drawn. The three samples therefore do not overlap. Table 5.1 gives their sizes and the number of transfers connected to them.

Table 5.1.: The three disjoint HuggingFace confirmation samples. The source contains 114,739 labelled targets. 39,332 pass the common activity and upper-tail rules before prior-run exclusions.

Rep.	Available	Prevalence	Controlled	Total	Incident edges
1	26,904	5,000	3,903	8,903	17,381,820
2	18,001	5,000	3,697	8,697	24,714,450
3	9,304	5,000	2,293	7,293	27,901,492

The two 32-role samples contain 8,903 and 8,697 target addresses, with 17,381,820 and 24,714,450 connected transfers. The 40-role sample contains 7,293 targets and 27,901,492 connected transfers. For the 40-role analysis, the graph is extended to the released neighbourhood of the selected targets so that triangles can be counted. This graph contains 93,327,990 transfers among 9,950,430 addresses and includes all triangles around a selected target that are present in the published dataset.

The exclusions leave fewer addresses for the 40-role controlled cohort. It contains 1,790 benign and 503 malicious addresses, and all 503 malicious addresses are externally owned accounts. This cohort still controls for activity, but it cannot be used to compare malicious contracts with malicious ordinary accounts.

For Real-CATS, 11,225 labelled criminal addresses have usable events and 10,871 belong to the nine categories that meet the minimum class size. The five retained layers contain

12,142,326 unique events after 65,954 repeated records are removed. The separate binary sample contains 3,067 criminal and 3,067 benign addresses with similar activity levels. None of these addresses appears in the HuggingFace label set. Real-CATS does not provide campaign identifiers, so the train and test sets can be separated by address but not by campaign.

5.2. RQ1: Incremental Signal of Temporal Motif Fingerprints

Adding local motif-role features increases average precision in all six HuggingFace comparisons. Three of these increases are statistically significant. Figure 5.1 compares the baseline, motif-only, and combined feature sets in the 40-role analysis. Table 5.2 shows the same comparison for all three samples. The addresses and data splits remain identical within each comparison, so only the feature set changes.

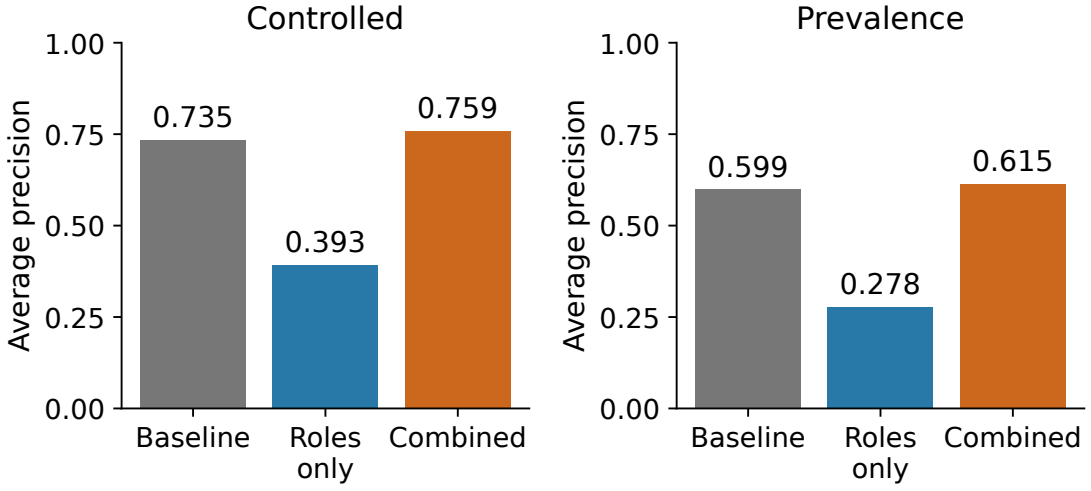


Figure 5.1.: RQ1 feature comparison in the 40-role HuggingFace analysis using the default timestamp ordering. The controlled cohort is shown on the left and the prevalence cohort on the right.

Table 5.2.: RQ1: average precision of each feature view at the selected window for three disjoint confirmations (canonical ordering), the combined-minus-baseline difference with its 95 % confidence interval, the permutation p-value, and the decision.

Rep.	Cohort	δ	Base	Motifs	Comb.	ΔAP	95 % CI	p	Decision
1	Controlled	2 weeks	0.891	0.739	0.903	0.013	[0.003, 0.023]	0.007	reject
1	Prevalence	30 days	0.756	0.526	0.789	0.033	[0.011, 0.056]	0.002	reject
2	Controlled	2 weeks	0.889	0.699	0.902	0.013	[0.002, 0.024]	0.021	reject
2	Prevalence	30 days	0.713	0.464	0.721	0.008	[-0.018, 0.033]	0.275	not rejected
3	Controlled	30 days	0.735	0.393	0.759	0.024	[-0.014, 0.062]	0.133	not rejected
3	Prevalence	30 days	0.599	0.278	0.615	0.016	[-0.034, 0.065]	0.266	not rejected

Three of the six improvements are statistically significant. Both cohorts are significant in one 32-role sample, while only the controlled cohort is significant in the other. Neither cohort is significant in the 40-role analysis. In that analysis, average precision rises from 0.7351 to 0.7587 in the controlled cohort. The difference is 0.0237, with a 95 % confidence

interval of $[-0.0142, 0.0615]$ and $p = 0.1330$. In the prevalence cohort, it rises from 0.5986 to 0.6146. Here the difference is 0.0161, with a confidence interval of $[-0.0342, 0.0653]$ and $p = 0.2659$. The direction remains positive under all seven timestamp orderings, but the confidence intervals include zero.

The motif-only model has lower AP than the non-motif baseline in every sample. The role-by-role fingerprint performs better than using only the total number of motifs. In the 40-role analysis, its AP advantage over motif volume is 0.1173 in the controlled cohort and 0.1528 in the prevalence cohort. Both differences remain significant after correction for multiple comparisons. Once the baseline is supplemented with motif composition and volume, adding the full role-by-role fingerprint does not give a significant improvement.

Table 5.3.: Prespecified secondary RQ1 representation comparisons at the development-selected window under canonical ordering. The first contrast tests fingerprint shape beyond total motif count. The second tests the combined model against a baseline augmented with motif composition and volume. P-values use Benjamini–Hochberg correction within the prespecified comparison family for each run.

Rep.	Cohort	Comparison	Ref.	Alt.	ΔAP	95 % CI	p_{BH}	Decision
1	Controlled	combined vs. augmented baseline	0.902	0.903	0.001	[-0.007, 0.010]	0.460	not rejected
1	Controlled	fingerprint vs. motif volume	0.434	0.739	0.306	[0.252, 0.351]	<0.001	reject
1	Prevalence	combined vs. augmented baseline	0.773	0.789	0.016	[-0.003, 0.036]	0.101	not rejected
1	Prevalence	fingerprint vs. motif volume	0.244	0.526	0.282	[0.220, 0.338]	<0.001	reject
2	Controlled	combined vs. augmented baseline	0.902	0.902	0.001	[-0.009, 0.010]	0.613	not rejected
2	Controlled	fingerprint vs. motif volume	0.421	0.699	0.278	[0.221, 0.325]	<0.001	reject
2	Prevalence	combined vs. augmented baseline	0.722	0.721	-0.001	[-0.019, 0.017]	0.862	not rejected
2	Prevalence	fingerprint vs. motif volume	0.230	0.464	0.234	[0.159, 0.301]	<0.001	reject
3	Controlled	combined vs. augmented baseline	0.750	0.759	0.008	[-0.014, 0.031]	0.464	not rejected
3	Controlled	fingerprint vs. motif volume	0.276	0.393	0.117	[0.026, 0.200]	0.034	reject
3	Prevalence	combined vs. augmented baseline	0.589	0.615	0.026	[0.000, 0.053]	0.069	not rejected
3	Prevalence	fingerprint vs. motif volume	0.126	0.278	0.153	[0.084, 0.233]	<0.001	reject

Table 5.4 examines the eight triangle roles separately. Adding them directly to the baseline changes AP by 0.0012 in the controlled cohort and by 0.0001 in the prevalence cohort. Using all 40 roles instead of the 32 star and two-node roles changes AP by 0.0071 and 0.0041, respectively. None of these differences is significant.

In the binary Real-CATS comparison, the selected window is 30 days. Baseline AP is 0.9344 and combined AP is 0.9356. The difference is 0.0011 with a confidence interval of $[-0.0018, 0.0042]$ and $p = 0.2440$. F1 decreases from 0.8481 to 0.8351. The positive AP direction is not preserved under every timestamp ordering. The motif-only model still reaches an AP of 0.8708.

For the XGBoost comparison, the AP difference is 0.0217 in the controlled cohort ($p =$

Table 5.4.: Final RQ1 triangle-role ablation at the selected 30-day window under canonical ordering. Corrected p-values use the prespecified representation-comparison family.

Cohort	Comparison	Ref.	Alt.	ΔAP	95 % CI	p_{BH}	Decision
Controlled	triangles added to baseline	0.735	0.736	0.001	[-0.020, 0.024]	0.540	not rejected
Controlled	40 roles versus 32 roles	0.752	0.759	0.007	[-0.011, 0.026]	0.464	not rejected
Prevalence	triangles added to baseline	0.599	0.599	0.000	[-0.025, 0.025]	0.547	not rejected
Prevalence	40 roles versus 32 roles	0.611	0.615	0.004	[-0.016, 0.024]	0.448	not rejected

Table 5.5.: Independent binary replication on the activity-matched Real-CATS sample under canonical ordering.

Quantity	Value
Addresses	6,134
Selected window	30 days
One-standard-error window	3 days
Baseline AP	0.934
Motif-only AP	0.871
Combined AP	0.936
ΔAP	0.001
95 % CI	[-0.002, 0.004]
Permutation p-value	0.244

0.1224) and −0.0119 in the prevalence cohort ($p = 0.7470$).

5.3. RQ2: Sensitivity to the Temporal Window

AP and fingerprint coverage generally increase over the tested window range. Figure 5.2 shows how held-out average precision and motif coverage change across the nine tested windows in the 40-role HuggingFace analysis. The baseline is fixed because its features do not depend on δ. Table 5.6 shows the windows selected on the development data in all three samples.

Table 5.6.: RQ2: development-CV window selections for three disjoint confirmations. The final column states whether the selected maximum was uniquely superior after corrected comparisons under all seven orderings.

Rep.	Cohort	Maximum δ	1-SE δ	Unique maximum
1	Controlled	2 weeks	1 week	no
1	Prevalence	30 days	1 day	no
2	Controlled	2 weeks	1 week	no
2	Prevalence	30 days	30 days	no
3	Controlled	30 days	2 weeks	no
3	Prevalence	30 days	2 weeks	no

Both 32-role controlled cohorts select two weeks, while the 40-role controlled cohort and all three prevalence cohorts select 30 days. In the 40-role analysis, 30 days is selected in both cohorts under all seven timestamp orderings, while two weeks remains within one standard error of the maximum. Mean development-CV AP increases from 0.7431 at five minutes to 0.7760 at 30 days in the controlled cohort, and from 0.5897 to 0.6253 in the

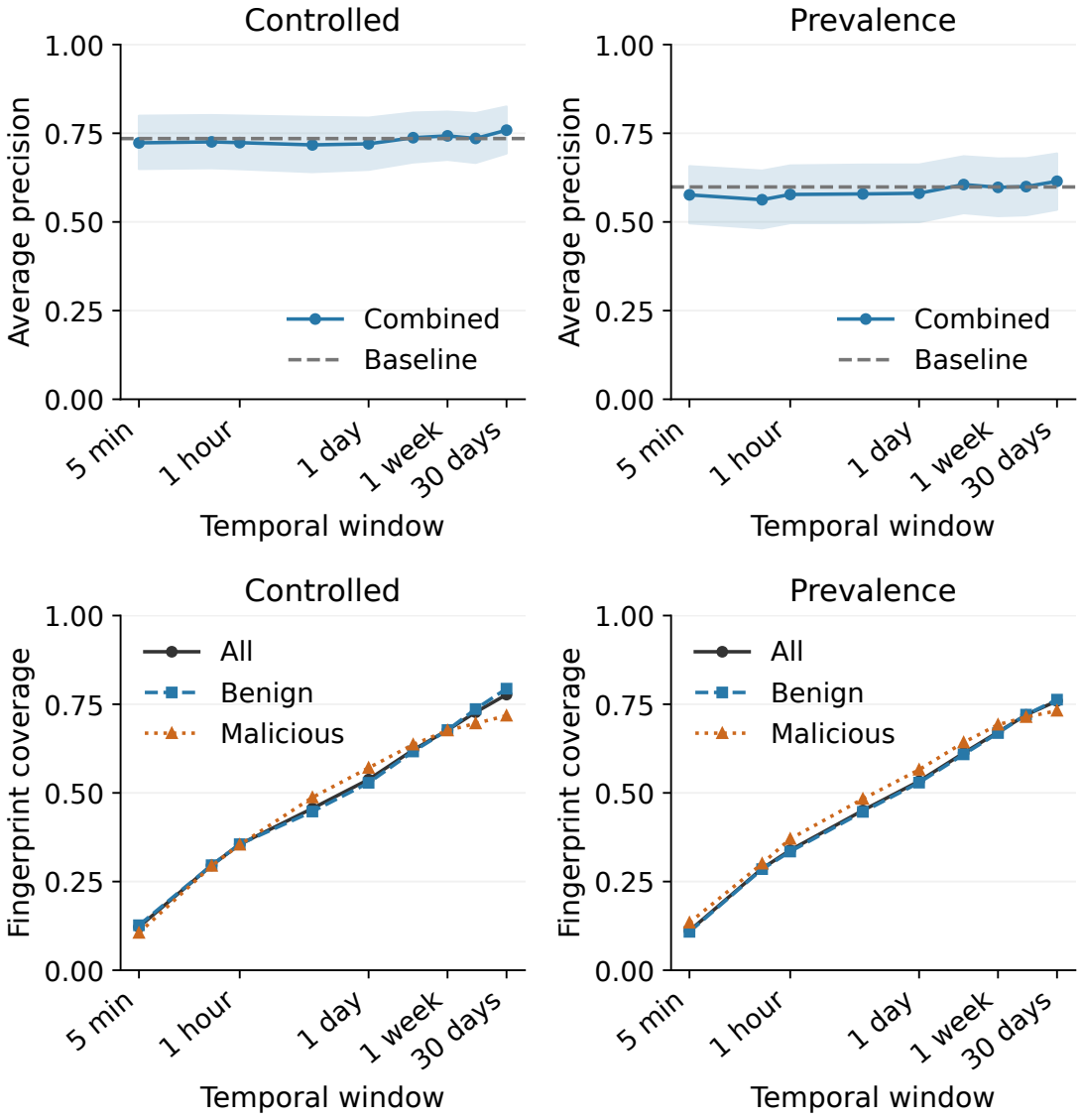


Figure 5.2.: RQ2 temporal-window sensitivity in the 40-role HuggingFace analysis using the default timestamp ordering. The controlled cohort is shown on the left and the prevalence cohort on the right. The top row reports held-out average precision with shaded 95 % bootstrap confidence intervals. The bottom row reports fingerprint coverage across all addresses in each cohort.

prevalence cohort. The binary Real-CATS comparison also selects 30 days, although three days is already within one standard error of its maximum.

After correction for multiple comparisons, none of the three HuggingFace samples has a selected maximum that is better than every other tested window under all orderings. In the 40-role controlled cohort, five of the eight comparisons with 30 days are significant using the default ordering. In the prevalence cohort, none is significant. The 30-day window is the largest tested value. It has the highest development score in the 40-role analysis, while two weeks is within one standard error of that maximum.

Fingerprint coverage increases from 12.2% at five minutes to 77.7% at 30 days in the 40-role controlled cohort. In the prevalence cohort, it increases from 11.1% to 75.9%. This means that 22.3% and 24.1% of the respective addresses still have no retained three-edge candidate pattern even at 30 days.

5.4. RQ3: Distinct Motif Fingerprints per Category

RQ3 evaluates category separability using motif fingerprints alone. It does not compare fingerprints with conventional behavioural features for category classification. The analysis contains 10,871 addresses across the categories listed in Table 3.2. Their distribution is very uneven: phishing accounts for most addresses, while the seven smallest categories have only 25 to 47 addresses each in the test set.

Table 5.7 reports the overall classification result, and Table 5.8 gives the result for each category separately. Figure 5.3 compares the average fingerprints, and Figure 5.4 reports the confusion matrix.

The heatmap labels use layer prefixes for ordinary transfers (N), internal transfers (I), ERC-20 (E20), ERC-721 (E721), and ERC-1155 (E1155). The labels pre, mid, and post identify the three star classes, and two denotes a two-node role. Direction codes use i for incoming and o for outgoing. Category labels in the figures omit the shared suffix Scam.

Table 5.7.: RQ3: overall separability of the criminal categories from their motif fingerprints (canonical ordering).

Quantity	**Value**
Global window	2 weeks
Macro average precision	0.195
Macro F1	0.191
Permutation p-value	<0.001
Decision	reject

Using the default timestamp ordering, the development data selects a shared window of two weeks. On the test set, macro AP is 0.1953 and macro F1 is 0.1914. For comparison, the average class share across the nine one-vs-rest tasks is $1/9 = 0.1111$. The permutation test gives $p < 0.001$. It rejects the label-permutation null under all seven timestamp orderings, although their selected windows range from one week to 30 days.

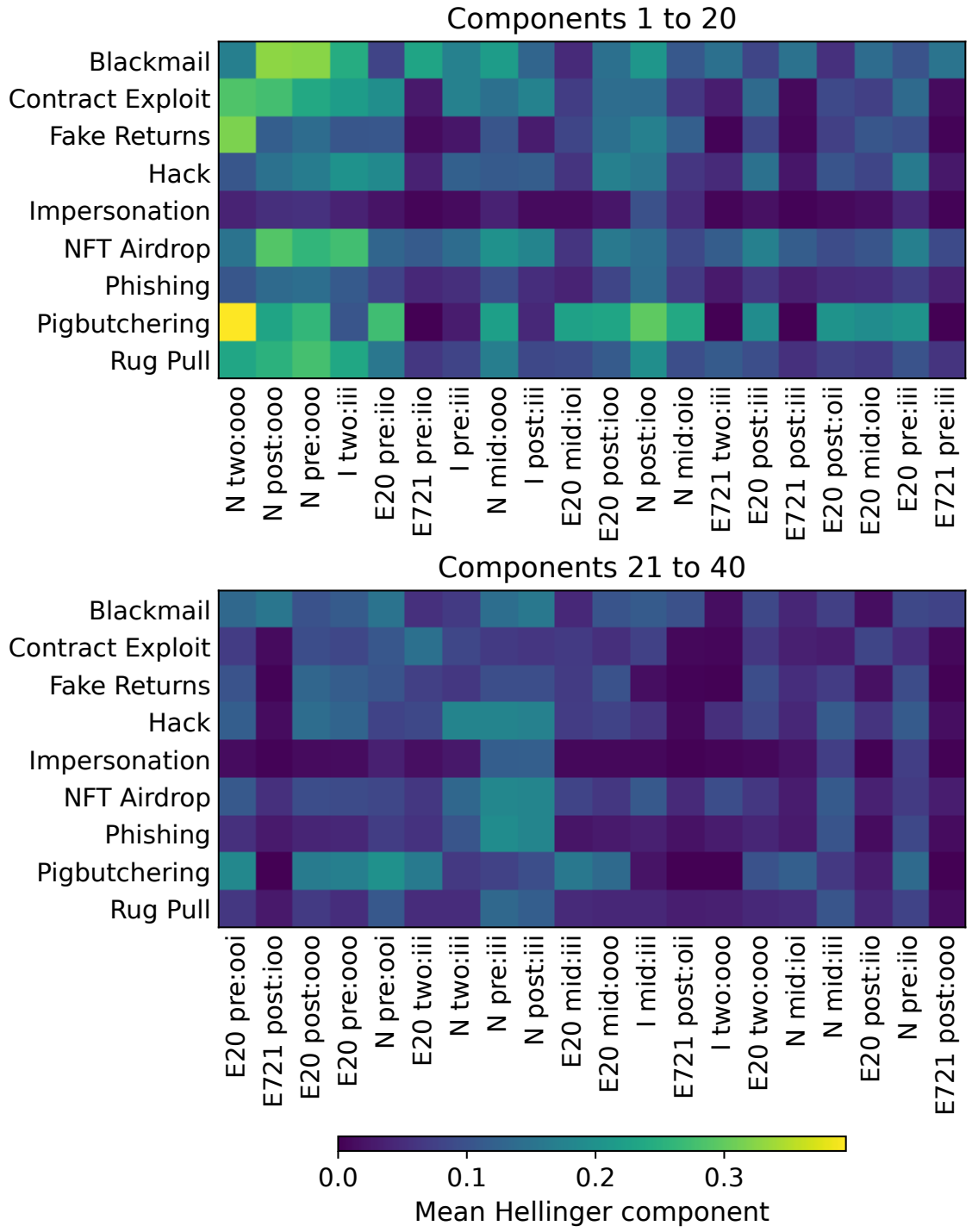


Figure 5.3.: RQ3 category fingerprints using the globally selected two-week window and the default timestamp ordering. The two panels show the 40 local role components that vary most between categories in the development data, divided into two groups of 20 for readability. These components are selected from the five layers of 32 roles each, not from the 40-role HuggingFace catalogue.

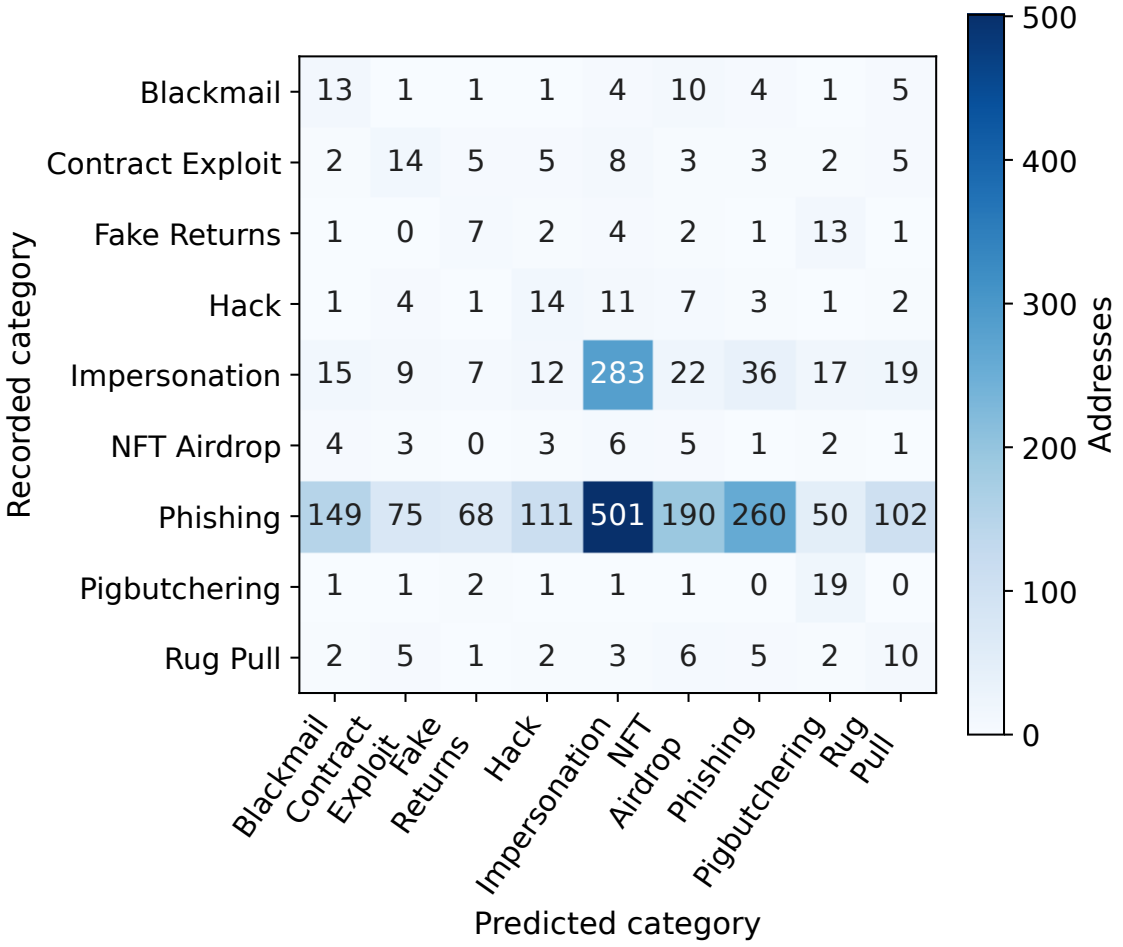


Figure 5.4.: RQ3 confusion matrix for the held-out Real-CATS addresses at the shared two-week window under the default timestamp ordering. Cells report numbers of addresses.

Table 5.8.: RQ3: one-vs-rest average precision for each behaviour category at the globally selected window (canonical ordering). The address count and class share are included because average precision depends on class prevalence.

Category	n	Share	AP
Phishing Scam	7,525	0.692	0.771
Impersonation Scam	2,099	0.193	0.320
Pigbutchering Scam	131	0.012	0.259
Contract Exploit Scam	234	0.022	0.116
Hack Scam	220	0.020	0.079
Fake Returns Scam	154	0.014	0.067
Rug Pull Scam	179	0.016	0.065
Blackmail Scam	202	0.019	0.056
NFT Airdrop Scam	127	0.012	0.024

The one-vs-rest AP values range from 0.0244 for NFT Airdrop Scam to 0.7710 for Phishing Scam. Every value is above the share of its category. Pigbutchering Scam reaches 0.2594 AP with a class share of only 0.0121. Phishing has the highest AP, but it also represents 69.22% of the sample. Table 5.8 reports the class shares alongside AP.

A model based on log-transformed local role counts reaches macro AP 0.2058 and macro F1 0.1868, compared with 0.1953 and 0.1914 for the composition-based fingerprint.

5.5. RQ4: Category-Specific Temporal Windows

The comparison of category-specific and shared windows gives a test-set macro AP difference of 0.0009, which is not statistically significant. The best window for each category is selected on the development data and then evaluated on the same held-out addresses as the shared two-week window. Table 5.9 reports the overall comparison, and Table 5.10 gives the result for each category. Figure 5.5 shows how the development score changes across the tested windows.

Table 5.9.: RQ4: held-out comparison of development-selected category-specific windows with one global window (canonical ordering).

Quantity	Value
Global window	2 weeks
Distinct category windows	6
Δ macro AP	0.001
95% CI	[-0.015, 0.014]
Bootstrap p-value	0.539
Decision	not rejected

The development data selects six different windows under the default ordering. They range from six hours for Blackmail Scam to 30 days for Phishing Scam. Three categories select one week, two select two weeks, and the remaining categories select one day or three days.

On the held-out test set, using the category-specific windows instead of the shared two-week window changes macro AP by only 0.0009. The 95 % confidence interval is $[-0.0153, 0.0139]$ and $p = 0.5386$. Rug Pull Scam and Blackmail Scam show the largest positive changes, at

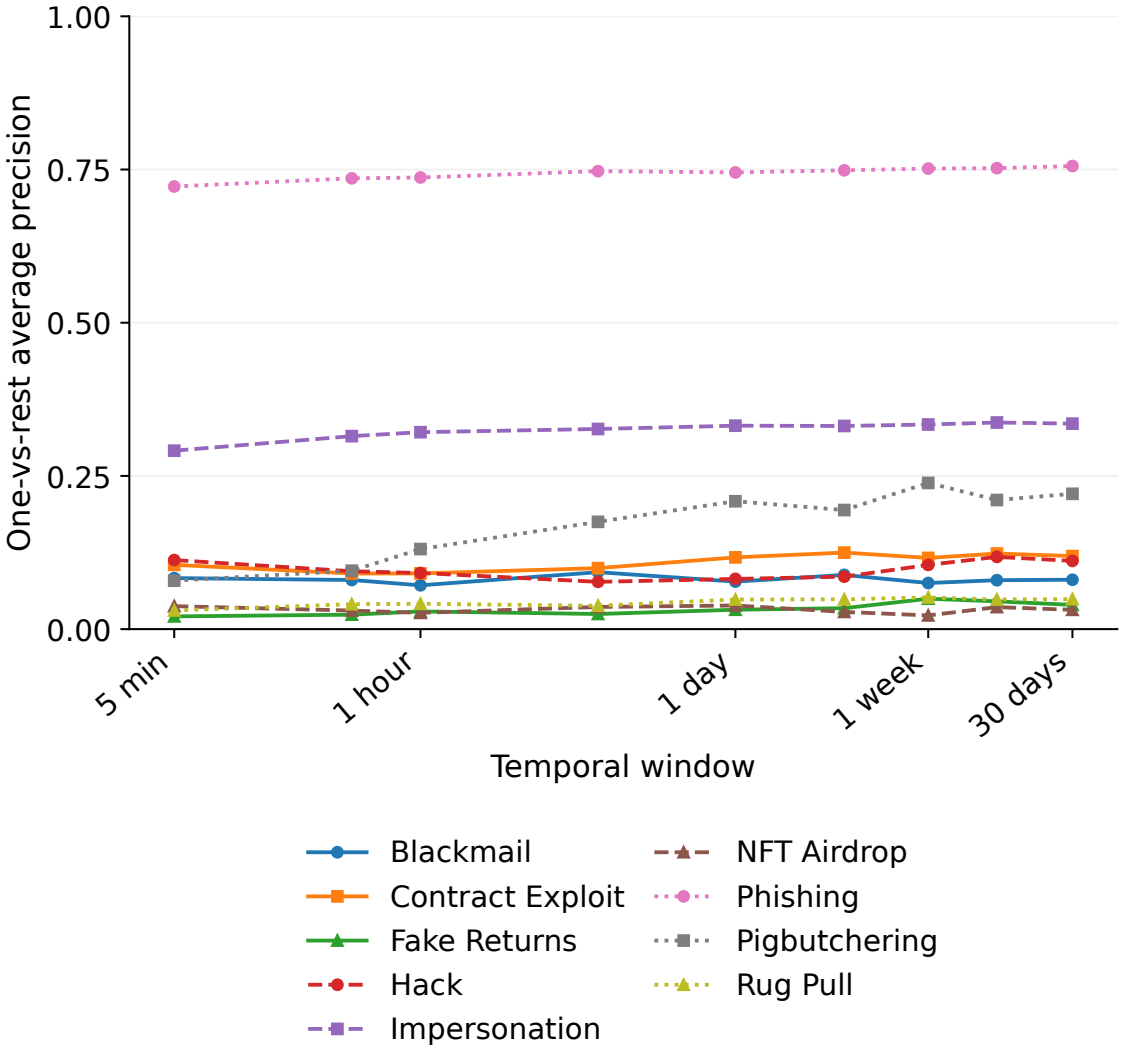


Figure 5.5.: RQ4: mean development-CV average precision for each category across the nine temporal windows using the default timestamp ordering.

0.0252 and 0.0230, but both confidence intervals include zero. None of the nine category-level differences remains significant after correction for multiple comparisons.

The selections also depend on how transactions with the same timestamp are ordered. Only Impersonation Scam and Pigbutchering Scam select the same window under all seven orderings. The selected window changes at least once for every other category. For example, Phishing Scam selects 30 days under six orderings but one week when the default tie order is reversed.

5.6. Additional Checks

Timestamp ordering.

Repeating the analysis with all seven ways of ordering transactions that share a timestamp does not change the overall answers. In the 40-role HuggingFace analysis, both RQ1 differences remain positive but non-significant, and the selected RQ2 window remains 30

Table 5.10.: RQ4: category-specific windows compared with the global window (canonical ordering). P-values use Benjamini–Hochberg correction.

Category	δ	ΔAP	95% CI	p_{BH}	Decision
Blackmail Scam	6 hours	0.023	[-0.020, 0.079]	1.000	not rejected
Contract Exploit Scam	3 days	-0.003	[-0.055, 0.042]	1.000	not rejected
Fake Returns Scam	1 week	-0.012	[-0.047, 0.005]	1.000	not rejected
Hack Scam	2 weeks	0.000	[0.000, 0.000]	1.000	not rejected
Impersonation Scam	2 weeks	0.000	[0.000, 0.000]	1.000	not rejected
NFT Airdrop Scam	1 day	-0.006	[-0.049, 0.012]	1.000	not rejected
Phishing Scam	30 days	-0.004	[-0.016, 0.007]	1.000	not rejected
Pigbutchering Scam	1 week	-0.015	[-0.114, 0.072]	1.000	not rejected
Rug Pull Scam	1 week	0.025	[-0.031, 0.084]	1.000	not rejected

days. The RQ3 result also remains significant. For RQ4, the comparison with the shared window remains non-significant, although most of the selected category windows change.

Subgraph counting.

The synthetic validation templates cover all 40 local role positions in Raphtory's output vector, including the eight triangle roles. The counter recovers all expected role counts and handles candidate patterns exactly at the temporal-window boundary correctly.

Optional BLTE dataset.

No completed BLTE result is included in this chapter. BLTE therefore contributes no evidence to the answers above and remains an optional secondary check only.

5.7. Answers to the Research Questions

Table 5.11 brings together the main result for each research question. The earlier sections provide the corresponding values and uncertainty.

Table 5.11.: Summary of the answers to the four research questions.

Question	Finding	Decision	Ordering check
RQ1	small positive AP increment	not supported in the final HF or external test	positive final HF direction in every ordering: yes
RQ2	30 days maximizes final development CV	unique superiority supported: no	same maximum in every final HF ordering: yes
RQ3	category fingerprints are distinguishable	supported	same decision in every ordering: yes
RQ4	category-specific windows improve test AP	not supported	stable test decision, unstable selected windows

6. Discussion

This thesis examined whether temporal motifs provide useful information for detecting malicious Ethereum addresses and for distinguishing between different categories of malicious behaviour. The results lead to one overall conclusion. Motif fingerprints contain information about address behaviour, but they do not reliably improve classification once conventional transaction and graph features are already available. The temporal window has a clear effect on how many motifs can be observed, but the experiments do not identify one unique window that is best for every sample. The Real-CATS categories also have detectable differences in their fingerprints, although these differences are too weak for accurate category classification. Selecting a separate window for each category does not improve this result.

These findings are discussed below in the same order as the research questions. The chapter then considers their theoretical and practical implications and the limitations of the study.

6.1. Interpretation of the Main Findings

6.1.1. Detection value of temporal motifs

RQ1 concerns the complete fingerprint representation. The comparison tests whether adding local motif-role features to the 17 conventional features improves detection. Since the roles combine structure, direction, focal-node position, and edge order, the result does not isolate a causal contribution of temporal ordering.

The first result is that temporal motifs contain a useful signal, but that their additional value depends on which other features are already present. Adding the motif fingerprint to the baseline increases AP in all six HuggingFace comparisons. Three of these increases are significant. The non-significant comparisons include the prevalence cohort of one 32-role sample and both cohorts of the 40-role analysis. In that analysis, the improvements are 0.0237 and 0.0161, but both confidence intervals include zero. The binary Real-CATS comparison is even closer, with an AP difference of only 0.0011. It also produces a lower F1 score after the local motif-role features are added. The positive direction is therefore not enough to conclude that motifs provide a reliable improvement over the baseline.

This does not mean that the motif fingerprint contains no information. The motif-only models can still rank addresses to some degree, and the Real-CATS motif-only model reaches an AP of 0.8708. More importantly, the role-by-role fingerprint performs better than total motif volume alone. In the 40-role HuggingFace analysis, this difference is 0.1173 AP in the controlled cohort and 0.1528 AP in the prevalence cohort. This shows that the distribution of activity across motif roles is more informative than the number of motifs by itself.

The result changes once conventional features are added. A baseline supplemented with motif composition and motif volume performs similarly to the full combined model across

all three HuggingFace samples. The detailed role counts therefore contain a signal, but much of that signal overlaps with information that can already be obtained from simpler summaries. This overlap is plausible because many transaction patterns affect several feature groups at the same time. A rapid fan-out, for example, changes the motif fingerprint, but it can also increase the number of outgoing transfers, the out-degree, and the number of transaction partners. The comparison does not isolate the exact source of this overlap, but it shows that the complete fingerprint adds little once these related properties are present.

The triangle analysis leads to the same interpretation. Extending the HuggingFace graph makes it possible to retain all eight triangle roles, but adding them changes AP by at most 0.0071 in the reported comparisons. None of these differences is significant. Triangle motifs may still be useful in other graphs or for other kinds of malicious behaviour. The present results only show that they do not strengthen address detection in these cohorts.

The strong baseline performance is consistent with earlier Ethereum detection work based on transaction aggregates and static graph features (Chen and Fu, 2022; Sanjalawe and Al-Emari, 2023). The motif-only results also agree with the observation of Arnold et al. (2024) that local motif signatures can expose behavioural differences that disappear in global counts. This thesis tests the next step by asking whether that information improves a detector on held-out addresses. The comparison shows that a descriptive signal does not automatically provide additional predictive value. The result also differs from the improvement reported by Liu et al. (2023) on non-blockchain financial networks. This suggests that results from one type of transaction network do not necessarily transfer to another, especially when the labels and baseline features are different.

Temporal motif features have also been used for Ethereum phishing detection (Wang et al., 2023). The present results qualify the usefulness of this representation under the labels and evaluation protocol used here. They do not provide a direct comparison with that earlier detector.

6.1.2. Choice of the temporal window

RQ2 asks how window size affects fingerprint usefulness. Over the tested range, longer windows generally increase both detection AP and the number of addresses with a nonzero fingerprint. This answers the question without requiring a unique optimum. The 40-role HuggingFace analysis selects 30 days for both cohorts, while two weeks remains within one standard error of the maximum. Both 32-role controlled cohorts select two weeks, and all three prevalence cohorts select 30 days. The Real-CATS binary comparison also selects 30 days, but a window of only three days is within one standard error of its maximum. These results point towards a broader range of longer windows rather than one clearly best setting.

Motif coverage helps explain this pattern. In the 40-role HuggingFace analysis, coverage increases from 12.2% to 77.7% in the controlled cohort and from 11.1% to 75.9% in the prevalence cohort between five minutes and 30 days. A short window leaves most addresses

with an all-zero fingerprint because their relevant transfers do not occur close enough together. Increasing δ allows more three-edge sequences to be counted and gives the model information for more addresses. The parallel increase in coverage and AP suggests that part of the benefit of a longer window comes from reduced sparsity. The experiment does not show how much of the improvement is caused by coverage and how much comes from a more suitable behavioural time scale.

This distinction is important for interpreting RQ2. A selected window defines the maximum time span of one three-edge motif. It does not measure the duration of a fraud campaign. A 30-day window can contain a rapid transaction sequence in the same way as a five-minute window, but it also admits sequences whose edges are much farther apart. The selected value therefore reflects a compromise between observing enough motifs and keeping the events temporally close. Since 30 days is the upper boundary of the tested grid, the results also cannot show whether performance would continue to increase at still larger windows. The supported conclusion is limited to the tested range.

6.1.3. Differences between behaviour categories

The category analysis uses motif fingerprints alone. Its class-share and permutation references establish whether the fingerprints contain category information. They do not establish whether this representation is more useful than conventional behavioural features for category classification, which would require an additional baseline.

RQ3 shows that the Real-CATS categories are not identical in their motif fingerprints. At the shared two-week window, the model reaches a macro AP of 0.1953 compared with an average class share of 0.1111. The permutation test finds the category signal under every timestamp ordering. This provides evidence that the motif distributions contain information about the recorded behaviour category.

The practical separation is much weaker than the permutation result alone might suggest. Macro F1 is 0.1914, and the confusion matrix shows that many addresses are assigned to the wrong category. The one-vs-rest AP values also need to be read together with their class shares. Phishing Scam has the highest AP, but it also represents more than two thirds of the sample. Several smaller categories have only 25 to 47 addresses in the test set, which makes their individual results less precise.

One likely reason for the limited separation is that a category label does not define one transaction pattern. Two phishing addresses can use different collection and withdrawal strategies, while an impersonation scam and a fake returns scam can produce similar transfers despite having different social engineering methods. The label describes the purpose of the malicious activity, whereas the motif fingerprint describes local transaction structure. These two descriptions are related, but they are not equivalent. The slightly higher AP of the count-based model suggests that motif volume may also contain category information. However, the comparison changes more than volume alone and does not identify which part of the count representation produces the difference.

RQ4 further limits the interpretation of the category results. The development data selects six different category-specific windows, ranging from six hours to 30 days. On the held-out test set, using these windows instead of the shared two-week window changes macro AP by only 0.0009. The confidence interval includes zero and none of the category-level differences is significant after correction for multiple comparisons. Most selected windows also change when transactions with the same timestamp are ordered differently.

The development maxima should therefore not be treated as stable properties of the categories. Selecting the highest point from nine windows will produce a maximum for every category even when several windows perform almost equally. The held-out comparison tests whether these apparent differences generalise, and in this study they do not. The category-specific windows also do not estimate how long the corresponding scam types last. They only describe where the classifier performed best on one development sample.

6.2. Theoretical Implications

One implication is that information content and incremental predictive value are not the same. Temporal motifs can describe address behaviour in a meaningful way even when they do not improve a classifier that already uses strong conventional features. The motif-only and representation results show that the role distribution contains information. The combined-model results show that this information is partly redundant with transaction aggregates and static graph properties. A representation can therefore be useful for describing a network without being the best way to improve detection performance.

The results also clarify what can be attributed to temporal ordering. Each motif role combines directed graph structure, the local position of the target address, and the order of the three edges. The advantage over motif volume shows that the detailed role distribution matters. It does not isolate temporal order from the other parts of the representation. A direct comparison with otherwise identical unordered motif features would be needed to measure the contribution of order on its own. The present comparison instead evaluates the complete temporal-motif representation against the non-motif baseline.

The results also clarify the meaning of δ. The window is often described as the time scale of a motif, but the results show that it also controls whether a fingerprint can be observed at all. At short windows, sparsity is the dominant problem for many addresses. At long windows, more sequences become available, but transactions that are less closely related can also be combined. This makes the window both a behavioural and a measurement parameter. The lack of a unique optimum suggests that malicious behaviour may not operate at one shared time scale. It also suggests that a representation combining several windows could be more suitable than forcing every address into one value of δ. This possibility is not tested in the present analyses.

The category results raise another issue. Broad fraud labels do not map directly to single network structures. Their fingerprints are distinguishable in aggregate, but the overlap between categories remains large. Temporal motifs therefore capture one part of malicious behaviour rather than a complete model of it. Off-chain intent, the role of smart contracts,

token type, and the wider campaign context may distinguish two categories even when their local transfer patterns look similar. Conversely, two addresses with the same category can have different roles within a campaign and therefore different fingerprints.

These points narrow the claim made in this thesis. Temporal motifs do retain structure that aggregate features cannot express directly. However, the existence of this additional representation does not guarantee a large classification gain. Its value depends on the baseline, the available graph, the address activity, and the way malicious behaviour is labelled.

6.3. Practical Implications

For binary malicious-address detection on the datasets used here, the conventional feature set remains the most practical starting point. It performs better than the motif-only model and requires less computation than a sweep over temporal windows and timestamp orderings. Adding motifs may provide a small improvement in some samples, but the effect is not stable enough to justify treating the combined model as generally superior.

The same cost consideration applies to triangles. Constructing the released neighbourhood closure substantially increases the number of transfers that must be processed. The resulting triangle roles do not provide a significant gain in the final HuggingFace experiment. For a similar address-level detection task, it may therefore be reasonable to begin with the 32 star and two-node roles and add triangles only when the graph supports them and their contribution can be tested. This is a conclusion about the current datasets rather than a recommendation to exclude triangles from every blockchain analysis.

If motifs are used, the temporal window should be selected on development data rather than chosen from an assumed scam duration. The one-standard-error results also provide a practical alternative to selecting the numerical maximum. A shorter window that performs nearly as well can reduce the amount of history needed for an address and may reduce the counting workload. The current study does not measure online latency, so it cannot determine the best operational trade-off.

The fingerprints may still be useful as descriptive summaries for an analyst. They show whether an address mainly participates in repeated two-node exchanges, stars, or triangles, and whether its role is mainly incoming or outgoing. Such a summary could help compare addresses or inspect a suspected campaign. This possible use is not evaluated here. The evaluation measures classification performance, not whether motif explanations help an investigator make better decisions.

The Real-CATS results should also not be used as a stand-alone category detector. The significant permutation result shows that category information exists, but the low macro F1 shows that the predictions are not accurate enough for that use. A practical system would need additional information beyond the motif fingerprint, especially for the smaller categories.

6.4. Limitations

Candidate patterns and network significance.

The experiments count candidate temporal subgraphs and evaluate their local role fingerprints as predictive features. No random-network reference is generated, so the study cannot establish that any individual pattern is a statistically overrepresented network motif. Synthetic templates validate the counter, and prediction permutations test classification results. Neither replaces a network null model.

Labels and unit of analysis.

The study treats the published address labels as ground truth. These labels are based on off-chain reports and investigations, which can be incomplete or incorrect. An address label also describes the address rather than a complete campaign. One campaign can use many addresses, and one address can perform only one part of the wider behaviour. The HuggingFace splits keep known campaign groups together where that information is available. Real-CATS does not provide a campaign identifier, so its train and test sets can be separated by address but not by campaign. Similar addresses from the same unknown campaign could therefore appear in both sets.

Category size and heterogeneity.

The Real-CATS category distribution is strongly imbalanced. Phishing Scam dominates the data, while several categories contain only a small number of test addresses. Macro metrics give every category equal weight, but they do not remove the uncertainty caused by small samples. The exact labels are also broad and can contain several transaction strategies. RQ3 therefore tests whether the recorded categories are distinguishable on average. It does not show that every address within one category shares a common motif fingerprint.

Graph coverage.

The HuggingFace triangle counts are complete relative to the published release and the released-neighbourhood closure, but not necessarily relative to the full Ethereum chain. Transactions absent from the release cannot be recovered by the pipeline. Real-CATS provides detailed per-address histories across five layers, but these histories do not guarantee the partner-to-partner edge required for a complete triangle. Its category analysis therefore uses only star and two-node roles. The two datasets consequently do not provide exactly the same view of the transaction graph.

Cohort composition.

The three HuggingFace samples use non-overlapping addresses and known campaign groups, which prevents their direct reuse between samples. Repeated exclusions also change the population that remains. The 40-role controlled cohort contains 503 malicious addresses and none of them is a contract account. Its result cannot establish whether local motif-role

features help detect malicious contracts. Two samples use 32 motif roles, while the remaining sample uses all 40. The analyses therefore test the same main question on different samples, but they are not identical replications of one feature definition.

Time representation.

Ethereum timestamps do not provide a unique order for every pair of transactions. Several transfers can share the same timestamp, especially when they occur in the same block. The seven ordering variants show that the main RQ1, RQ2, and RQ3 answers do not depend on one arbitrary tie order. They also show that most RQ4 window selections are not stable. The variants cannot recover the true order of events when the source data does not record it.

The window grid is another limitation. It covers values from five minutes to 30 days, and 30 days is selected in the 40-role HuggingFace and binary Real-CATS analyses. Since this value lies at the boundary, the study cannot determine whether a still larger window would perform better. The fixed grid also does not test every possible duration between its nine values.

Retrospective evaluation.

The task classifies an address after observing the transaction history available in the dataset. It does not measure how early a malicious address could be detected or how performance changes as the history grows. A longer window may be useful in this retrospective setting while being too slow for an operational warning system. The results should therefore not be interpreted as real-time detection performance.

Models and external validation.

The primary binary model is a random forest with fixed settings. XGBoost is used as a robustness check, and the category task uses logistic regression. These choices keep the comparison focused on the features, but they do not cover every possible classifier or interaction between model and representation. A model designed specifically for temporal graphs may use the motif information differently.

The binary Real-CATS sample provides an external comparison, but its labels and data construction differ from HuggingFace. BLTE remains optional and no completed BLTE result is included. The evidence for generalisation is therefore limited to one additional dataset. Results on other chains, time periods, and label sources may differ.

Taken together, these limitations do not remove the observed motif signal, but they restrict what can be concluded from it. The study supports temporal motifs as a meaningful description of address behaviour. It does not show a reliable improvement over conventional detection features, a universal temporal window, or a practical stand-alone classifier for behaviour categories.

7. Conclusion & Outlook

7.1. Conclusion

Malicious activity on Ethereum is difficult to detect from individual transactions. An address can look ordinary on its own even when it is part of a larger sequence involving several addresses. This thesis examined whether temporal motifs make such patterns useful for detection. Each address was represented by the roles it occupies in ordered three-edge patterns that occur within a temporal window δ. This representation was then compared with conventional transaction aggregates and static graph features.

The main contribution is an evaluation of temporal motif fingerprints on labelled Ethereum addresses. The evaluation covers binary detection on the HuggingFace Ethereum Fraud Dataset and a separate Real-CATS sample, as well as category classification across nine Real-CATS behaviour labels. The pipeline counts the retained motif roles for several temporal windows, keeps the five Real-CATS transaction layers separate, and accounts for transfers that share a timestamp. All main comparisons use held-out addresses. This makes it possible to test whether a visible motif signal also improves prediction on addresses that were not used to select the model or temporal window. The available labels identify malicious addresses and reported behaviour categories rather than independently verified coordinated campaigns. The evaluation therefore measures address-level behavioural signal, not direct campaign detection.

The answer to RQ1 is that temporal motifs contain information, but do not reliably improve malicious-address detection beyond the conventional baseline. Adding the fingerprint increases average precision in all six HuggingFace cohort comparisons, although only three of these increases are statistically significant. In the 40-role HuggingFace analysis, the increases are 0.0237 in the controlled cohort and 0.0161 in the prevalence cohort, with both confidence intervals including zero. The binary Real-CATS comparison produces an increase of only 0.0011 and a lower F1 score. At the same time, the role-by-role fingerprint performs better than motif volume alone. Its advantage in the 40-role analysis is 0.1173 average precision in the controlled cohort and 0.1528 in the prevalence cohort. The way an address participates in motifs is therefore more informative than the total number of motifs, but much of this information overlaps with simpler address features.

For RQ2, increasing the temporal window generally gives higher average precision and provides a non-zero fingerprint for more addresses. In the 40-role analysis, motif coverage rises from 12.2% to 77.7% in the controlled cohort and from 11.1% to 75.9% in the prevalence cohort between five minutes and 30 days. The 30-day window gives the highest development score in both cohorts, but two weeks remains within one standard error of the maximum. For no HuggingFace sample is the selected maximum significantly better than every other tested window under all timestamp orderings. Since 30 days is also the largest tested value, it cannot be treated as a universal optimum, and no unique optimum within the tested grid is established either. The selected window describes the maximum span of a counted candidate pattern, not the duration of a malicious campaign.

RQ3 shows that the nine Real-CATS behaviour categories have detectable differences in their motif fingerprints. At the shared two-week window, the classifier reaches a macro average precision of 0.1953, compared with an average class share of 0.1111. The permutation result remains significant under all seven timestamp orderings. However, the macro F1 score is only 0.1914 and the categories overlap strongly in the confusion matrix. The fingerprints therefore contain category information, but they are not sufficient for accurate category classification on their own. RQ3 establishes separability within this representation, not superiority over conventional behavioural features for category classification.

The answer to RQ4 is no. Replacing the shared two-week window with separately selected category windows changes held-out macro average precision by only 0.0009. The confidence interval includes zero, and none of the category-level changes is significant after correction for multiple comparisons. Most of the selected windows also change when equal-timestamp transfers are ordered differently. These development selections should not be interpreted as stable time scales of the corresponding behaviour categories.

The central result of the thesis is therefore a distinction between behavioural information and additional predictive value. Temporal motif fingerprints reveal how an address participates in local, ordered transaction patterns. This signal is visible in motif-only detection and in the comparison between Real-CATS categories. It does not consistently improve a detector that already has access to strong transaction aggregates and static graph features. Temporal motifs are therefore useful as a description of address behaviour, but the results do not support using them as a replacement for conventional features or as a generally reliable improvement to them.

7.2. Outlook

The existing analysis keeps the five Real-CATS transaction layers separate, but it counts motifs only within each layer. It therefore cannot represent one ordered pattern whose edges cross from an ordinary transfer to an internal transfer and then to a token transfer. The HuggingFace graph is limited to ordinary transaction edges as well. This matters for behaviour that is carried out through smart contracts, including contract exploits, token-based scams, and contract-mediated Ponzi schemes. A useful next step is a typed, multi-layer temporal graph in which the transaction type remains part of each motif. This would show whether cross-layer sequences provide information beyond the separate layer fingerprints already used in this thesis.

The temporal window also deserves a different treatment. The results suggest a broad range of useful longer windows rather than one stable optimum. A multi-scale fingerprint could combine short, medium, and long windows so that a rapid sweep and a slower consolidation pattern are not forced into the same value of δ. The tested grid could also be extended beyond 30 days, since its upper boundary is selected in several analyses. Any larger or multi-scale representation would need to be selected on development data and compared with the single-window fingerprint on held-out addresses. Otherwise, the additional windows could increase model complexity without improving generalisation.

A further experiment should separate temporal order from graph structure more directly. The current role catalogue combines direction, local node position, topology, and event order. It can show that the complete fingerprint is useful, but not how much of that usefulness comes from the order alone. Comparing it with an otherwise identical unordered representation would help isolate the contribution of temporal ordering. A separate random-network comparison could shuffle timestamps while retaining the directed transfers, then compare the candidate counts with their reference distributions. That experiment would test overrepresentation conditional on the preserved graph structure, with correction for the number of tested patterns and windows.

The current evaluation is retrospective. Every address is classified from the history available in the dataset, so the results do not show how quickly a new malicious address could be identified. A prospective evaluation could reveal transactions gradually and produce a prediction after fixed time intervals or after each new transfer. Detection performance could then be reported together with the time and number of transactions needed before an address is detected. This would provide a more useful test for an operational warning system than classification from a complete history.

External validation should also be extended. Real-CATS provides an independent binary comparison, but its labels and graph construction differ from the main HuggingFace data. BLTE remains an optional check and its transaction-level labels do not provide a direct equivalent to the address-level task. Future work would benefit from additional address-labelled datasets from other time periods and chains. Campaign identifiers would be especially valuable because they would allow related addresses to remain together during evaluation and would make it possible to study coordinated behaviour at campaign level rather than inferring it from individual address labels.

Finally, the model comparison can be extended. A temporal graph model may be able to use the underlying event sequence more directly than a random forest or logistic regression. The fingerprint may also be useful as a compact explanation for an analyst, even when it adds little to average precision. Both uses require separate evaluation. A more complex model should be compared with the same conventional baseline, and an explanatory use should be tested with analysts rather than inferred from classification scores.

These extensions follow directly from the main finding. Temporal motifs do capture meaningful transaction structure, but that structure only becomes valuable for detection when it adds something that simpler features do not already provide. Future work should therefore focus not only on counting more motifs, but on identifying which temporal and cross-layer patterns contribute new information on unseen addresses.

Appendix

A. Engineering challenges

The final design came out of an audit of the earlier exploratory pipeline against what the four research questions actually need. The audit turned up both speed bottlenecks and risks to the validity of the results, and the decisions below address them.

A.1. Locating the bottleneck

The largest data problem is the size of the HuggingFace source, which is never held in memory as a whole. The preparation phase scans the locally cached table in small batches and measures how active each target is. It writes the target-incident transfers and then performs a second scan for the released-neighbourhood closure needed by triangles. This keeps the memory demand bounded while making the target triangle counts complete relative to the release. Keeping the source locally removes download time and network failures, but does not remove the complete local scans or the larger closure graph.

Timing the earlier pipeline showed that reading the data and fitting the models were not the main cost. The dominant operation is Raphtory's local temporal motif routine. The complete HF protocol requires nine windows under seven timestamp orderings, and Real-CATS repeats the same grid within five layers. The graph is therefore built once per ordering (and, for Real-CATS, per layer and ordering) and reused across its nine windows, instead of being rebuilt for each window.

A.2. Alternatives considered

Three possible reductions were considered. First, counting only the motif roles suggested by the scam examples would appear to save work. Raphtory 0.17.0, however, exposes the 40-role counter as one operation and cannot be instructed to compute only selected roles. The pipeline consequently verifies the full output. HuggingFace retains all roles and evaluates the 32-role subset as an ablation. Real-CATS discards triangle columns after counting because its graph scope does not support complete target triangles. Avoiding those calculations would require a separate verified Rust implementation and was not introduced during this study.

Second, replacing pandas with Spark would not accelerate the actual counter, because the Raphtory graph still lives in one process. Pandas is used only after aggregation, for address-level targets, fingerprints, model matrices, predictions, and metrics. PyArrow performs bounded source scans and DuckDB performs the baseline query and external event sorting. A distributed dataframe rewrite would add another execution system without removing the single-process motif bottleneck.

Third, reducing the transaction types was considered. The HF graph already contains ordinary transaction edges only. For RQ3 and RQ4, in contrast, the separation of normal,

internal, ERC-20, ERC-721, and ERC-1155 histories is part of the question: an exploit or rug pull may differ precisely in which layer carries its activity. Dropping layers would change the fingerprint definition. The five layers are therefore kept, but loaded sequentially so that only one layer graph sets the Real-CATS peak memory demand.

A.3. Long-run execution

The workflow runs on a single machine, because the Raphtory graph has to live in one process. A read-only preflight estimates graph expansion, DuckDB memory, model matrices, checkpoint staging, sort spill, and a per-filesystem reserve. This is an admission budget rather than an enforceable Raphtory memory limit. If the budget does not fit, execution stops before creating an attempt instead of quietly shrinking the locked cohort.

Each expensive motif call is committed independently, at ordering–window level for HF and layer–ordering–window level for Real-CATS. A resumed execution rechecks a canonical content hash and reuses valid commits. An invalid committed checkpoint stops the run and is never replaced. HF and Real-CATS can also run in parallel on one server through a coordinator that gives them disjoint thread allocations and admits their combined RAM and disk demand. If one worker fails, the other is allowed to finish. These changes do not reduce the number of motif calculations in the scientific protocol, but they prevent an interruption late in a week-long run from discarding earlier verified work.

References

Al-Emari, S., Anbar, M., Sanjalawe, Y., and Manickam, S. (2021). A labeled transactions-based dataset on the Ethereum network. In Anbar, M. et al., editors, *Advances in Cyber Security (ACeS 2020)*, volume 1347 of *Communications in Computer and Information Science*, pages 61–79, Singapore. Springer Nature Singapore. National Advanced IPv6 Centre of Excellence (NAv6), Universiti Sains Malaysia (USM); dataset at https://github.com/salam-ammari/Labeled-Transactions-based-Dataset-of-Ethereum-Network.

Alomari, A. and Cheng, X. (2023). Probabilistic temporal sampling for anomaly detection in Ethereum transaction networks. In *OpenReview*.

Arnold, N. A., Zhong, P., Ba, C. T., Steer, B., Mondragon, R., Cuadrado, F., Lambiotte, R., and Clegg, R. G. (2024). Insights and caveats from mining local and global temporal motifs in cryptocurrency transaction networks. *Scientific Reports*, 14:26569.

Bartoletti, M., Carta, S., Cimoli, T., and Saia, R. (2020). Dissecting Ponzi schemes on Ethereum: Identification, analysis, and impact. *Future Generation Computer Systems*, 102:259–277.

Buterin, V. (2014). Ethereum: A next-generation smart contract and decentralized application platform. Whitepaper. https://ethereum.org/whitepaper/.

Chainalysis (2025). The Chainalysis 2025 crypto crime report. Technical report, Chainalysis Inc. Reports $40.9 billion in illicit on-chain volume for 2024; annual estimates have grown by an average of 25% between reporting periods since 2020.

Chen, H., Pendleton, M., Njilla, L., and Xu, S. (2020a). A survey on Ethereum systems security: Vulnerabilities, attacks, and defenses. *ACM Computing Surveys*, 53(3):1–43.

Chen, L., Peng, J., Liu, Y., Li, J., Xie, F., and Zheng, Z. (2020b). Phishing scams detection in Ethereum transaction network. *ACM Transactions on Internet Technology*, 21(1):1–16.

Chen, Y. and Fu, Z. (2022). Phishing scam detection on Ethereum via mining trading information. *Journal of Cyber Security*, 4(3):189–200. Engineering Research Center of Digital Forensics, Ministry of Education, Nanjing University of Information Science and Technology.

fesevu (2025). Ethereum fraud dataset by activity. https://huggingface.co/datasets/fesevu/ethereum_fraud_dataset_by_activity. Accessed: 2025-08-01.

Ghosh, M., Srivastava, S., Upadhyaya, A., Halder, R., and Chandra, J. (2025). CATALOG: Exploiting joint temporal dependencies for enhanced phishing detection on

Ethereum. In *Proceedings of the ACM Web Conference 2025 (WWW '25)*. ACM.

Hornuf, L., Momtaz, P. P., Nam, R. J., and Yuan, Y. (2025). Cybercrime on the Ethereum blockchain. *Journal of Banking & Finance*, 175:107419.

Kovanen, L., Karsai, M., Kaski, K., Kertész, J., and Saramäki, J. (2011). Temporal motifs in time-dependent networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2011(11):P11005. Department of Biomedical Engineering and Computational Science, Aalto University School of Science.

Lal, B., Agarwal, R., and Shukla, S. K. (2021). Understanding money trails of suspicious activities in a cryptocurrency-based blockchain. *arXiv preprint arXiv:2108.11818*.

Lee, X. T., Khan, A., Sen Gupta, S., Ong, Y. H., and Liu, X. (2020). Measurements, analyses, and insights on the entire Ethereum blockchain network. In *Proceedings of The Web Conference 2020 (WWW '20)*, pages 155–166. ACM.

Legendre, P. and Gallagher, E. D. (2001). Ecologically meaningful transformations for ordination of species data. *Oecologia*, 129(2):271–280.

Li, S., Zhou, J., Mo, C., Li, J., Tso, G. K. F., and Tian, Y. (2023). Motif-aware group signed temporal GCN for anomaly detection in bitcoin trust networks. *arXiv preprint arXiv:2211.13123v2*.

Li, T., Shin, D., and Wang, B. (2021). Cryptocurrency pump-and-dump schemes. In *SSRN Working Paper*.

Lin, D., Wu, J., Yuan, Q., and Zheng, Z. (2020). Modeling and understanding Ethereum transaction records via a complex network approach. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 67(11):2737–2741.

Liu, P., Altun, B., Acharyya, R., Tillman, R. E., Kimura, S., Masuda, N., and Sarıyüce, A. E. (2023). Temporal motifs for financial networks: A study on Mercari, JPMC, and Venmo platforms. *arXiv preprint arXiv:2301.07791*.

McDonnell, M. D., Yaveroğlu, Ö. N., Schmerl, B. A., Iannella, N., and Ward, L. M. (2014). Motif-role-fingerprints: The building-blocks of motifs, clustering-coefficients and transitivities in directed networks. *PLOS ONE*, 9(12):e114503.

Milo, R., Shen-Orr, S., Itzkovitz, S., Kashtan, N., Chklovskii, D., and Alon, U. (2002). Network motifs: Simple building blocks of complex networks. *Science*, 298(5594):824–827.

Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system. `https://bitcoin.org/bitcoin.pdf`. Accessed: 2026.

Newman, M. E. J. (2010). *Networks: An Introduction*. Oxford University Press.

Ostapowicz, M. and Żbikowski, K. (2019). Detecting fraudulent accounts on blockchain: A supervised approach. In *Web Information Systems Engineering – WISE 2019*, volume 11881 of *Lecture Notes in Computer Science*, pages 18–31. Springer.

Paranjape, A., Benson, A. R., and Leskovec, J. (2017). Motifs in temporal networks. In *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining (WSDM 2017)*, pages 601–610, Cambridge, United Kingdom. ACM.

Sanjalawe, Y. K. and Al-Emari, S. R. (2023). Abnormal transactions detection in the Ethereum network using semi-supervised generative adversarial networks. *IEEE Access*, 11:98516–98531. Received 17 August 2023; accepted 4 September 2023.

Shi, J., Duan, C., Lei, H., and Wang, L. (2025). Real-CATS: A practical training ground for emerging research on cryptocurrency cybercrime detection. *arXiv preprint arXiv:2501.15553*. Southeast University; dataset available at `https://github.com/sjdseu/Real-CATS`.

Wang, Y., Wang, H., Lu, X., Zhou, L., and Liu, L. (2023). Detecting Ethereum phishing scams with temporal motif features of subgraph. In *2023 IEEE Symposium on Computers and Communications (ISCC)*, pages 631–636. IEEE.

Weber, M., Domeniconi, G., Chen, J., Weidele, D. K. I., Bellei, C., Robinson, T., and Leiserson, C. E. (2019). Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics. *arXiv preprint arXiv:1908.02591*.

Wittek, K., Wittek, N., and Pohlmann, N. (2019). Statistical anomaly detection in Ethereum transaction graphs. *Proceedings of the 17th Annual Conference on Privacy, Security and Trust (PST)*.

Wu, J., Liu, J., Chen, W., Huang, H., Zheng, Z., and Zhang, Y. (2022a). Detecting mixing services via mining Bitcoin transaction network with hybrid motifs. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 52(4):2237–2249.

Wu, J., Yuan, Q., Lin, D., You, W., Chen, W., Chen, C., and Zheng, Z. (2022b). Who are the phishers? Phishing scam detection on Ethereum via network embedding. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 52(2):1156–1166. Introduces trans2vec; preprint arXiv:1911.09259 (2019).

Xia, P., Wang, H., Zhang, B., Ji, R., Gao, B., Wu, L., Luo, X., and Guo, Y. (2021). Trade or trick? Detecting and characterizing scam tokens on uniswap decentralized exchange. In *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, volume 5.

Yin, H., Wang, K., Zhang, W., He, Y., Zhang, Y., and Lin, X. (2025). Motif counting in complex networks: A comprehensive survey. *arXiv preprint arXiv:2503.19573*.